Technical Case Study

Document Intelligence

A Multi-Tenant Retrieval-Augmented Generation Platform

Java 17 · Spring Boot · React · PostgreSQL · Qdrant · Google Gemini

Kushal Upadhyay

github.com/kushal1-del · linkedin.com/in/kushalupadhyay108 · kushalupadhyay11@gmail.com · +91 85910 78291

01

Project at a Glance

Document Intelligence is a full-stack, multi-tenant RAG platform. Users upload PDF, DOCX and TXT files, and ask natural-language questions that are answered strictly from the content of their own documents — with the exact source chunks cited alongside every answer.

Hybrid
dense + sparse retrieval fused with RRF

Multi-tenant
owner-scoped isolation at the vector layer

BYOK
bring your own encrypted Gemini key

Layer	Technology
Frontend	React + Vite, Axios
Backend	Java 17, Spring Boot, Spring Security
Relational data	PostgreSQL
Vector store	Qdrant (dense + sparse hybrid collections)
Generation & embeddings	Google Gemini (768-dim embeddings, Gemini 3.6 Flash)
Auth	JWT, BCrypt
Secrets	AES-256-GCM encryption for stored API keys
Retrieval	Dense + sparse hybrid search, Reciprocal Rank Fusion (RRF)
Build	Maven

02

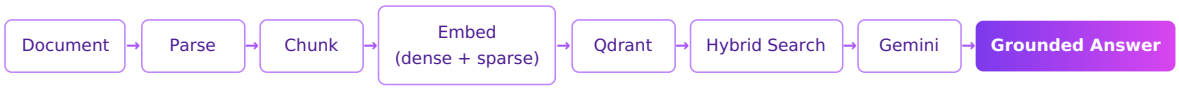
Problem → Solution

Problem

Keyword search over long documents forces people to guess the right words, and it can't answer a question that spans several paragraphs written in different phrasing. It returns pages, not answers, and it gives no way to verify that a claim is actually grounded in the source text.

Solution

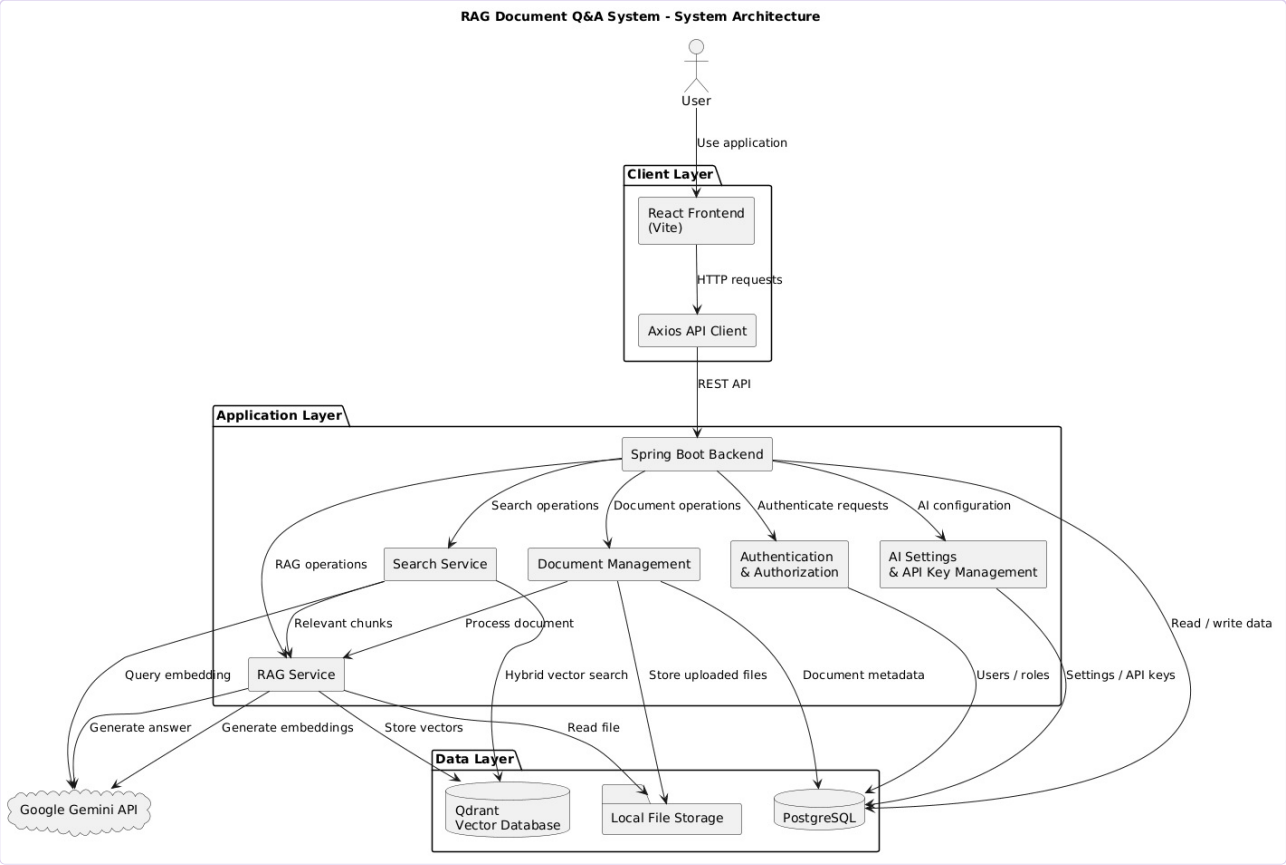
Every uploaded document is parsed, chunked, and embedded into both a dense semantic vector and a sparse lexical vector. A question is embedded the same way, matched against a user's own chunks with hybrid search, and the retrieved context is handed to Gemini with an explicit instruction to answer only from what was retrieved — with the matching chunks returned alongside the answer for verification.



03

System Architecture

The application follows a conventional three-layer split. The React frontend never talks to Qdrant, Gemini, or PostgreSQL directly — every request goes through the Spring Boot application layer, which owns authentication, tenant isolation, and orchestration across the two databases and the external AI provider.



System Architecture — client, application and data layers

> React (Vite) talks to Spring Boot over a REST API via Axios. Spring Boot fans out to Search, Document Management, Auth, and AI Settings services, which in turn read and write PostgreSQL (users, document metadata, settings, encrypted keys), Qdrant (vectors), local file storage (raw uploads), and the Google Gemini API (embeddings and generation).

04

The Complete RAG Pipeline

Two independent flows meet at Qdrant: an **ingestion** flow that runs once per document, and a **retrieval** flow that runs on every question.

Ingestion

1. Parse PDF / DOCX / TXT via a pluggable `FileParser` strategy
2. Chunk text with sentence-aware overlap (adaptive size for large documents)
3. Generate dense embeddings in batches via Gemini
4. Generate sparse (term-frequency + hashed) vectors locally
5. Upsert both vectors + payload (ownerId, documentId, chunkIndex, content) to Qdrant

Retrieval

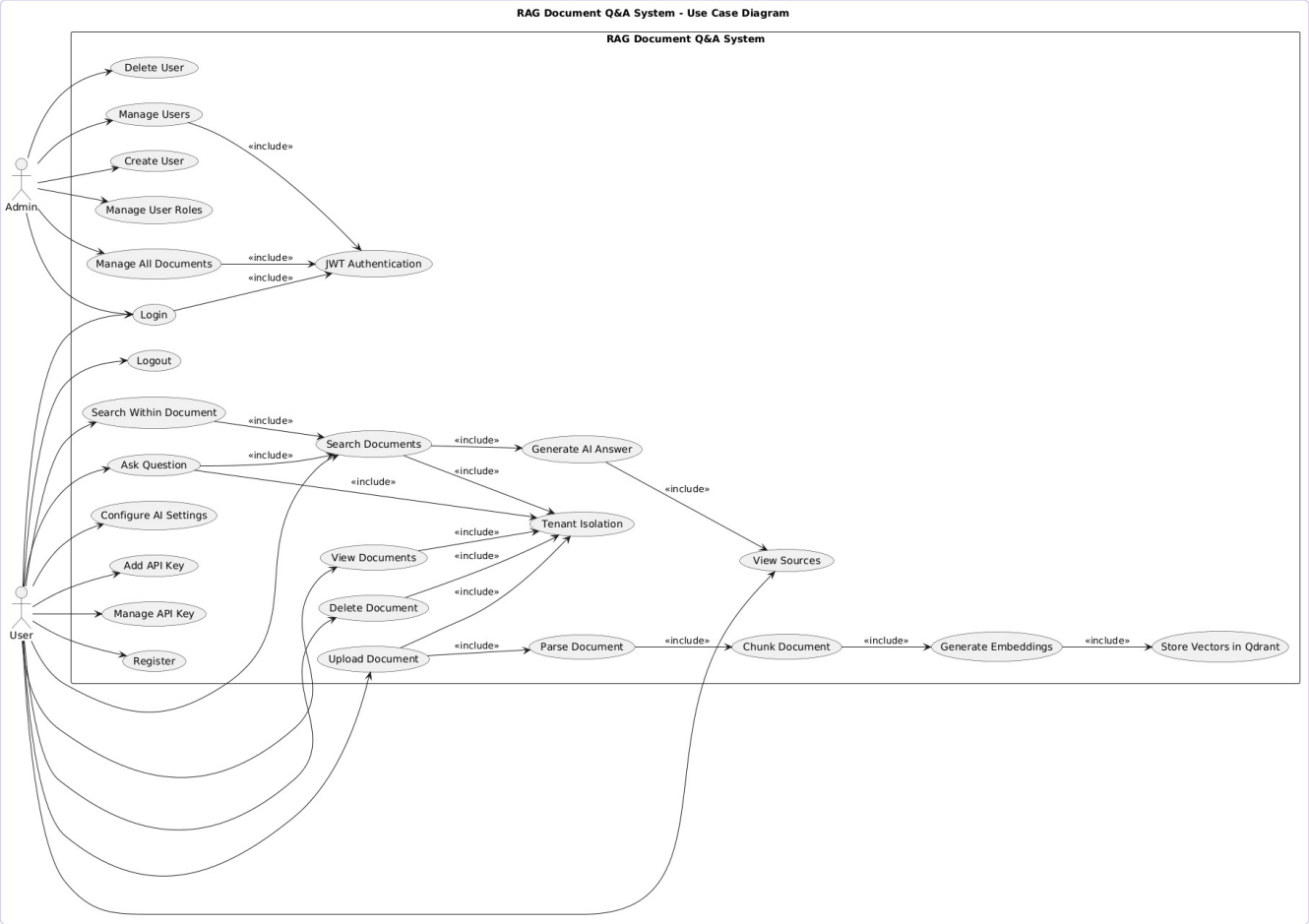
1. Embed the question (dense + sparse)
2. Run dense and sparse prefetch queries in parallel, filtered by ownerId (and optional documentId)
3. Fuse both rankings with Reciprocal Rank Fusion
4. Build a context string from the top-K chunks
5. Send context + question to Gemini with a grounding instruction; return answer + source chunks

> **Why hybrid search?** Dense vectors capture semantic meaning ("founder" ~ "started the company"), while sparse vectors catch exact keyword and identifier matches that embeddings can blur together. RRF combines both rankings without needing to tune a blend weight by hand.

05

Use Case Diagram

Two actors drive the system: an Admin who manages users and can moderate any document, and a User who owns a private document collection. Every data-touching use case includes Tenant Isolation and JWT Authentication, and the document pipeline is explicitly modelled as parse → chunk → embed → store.



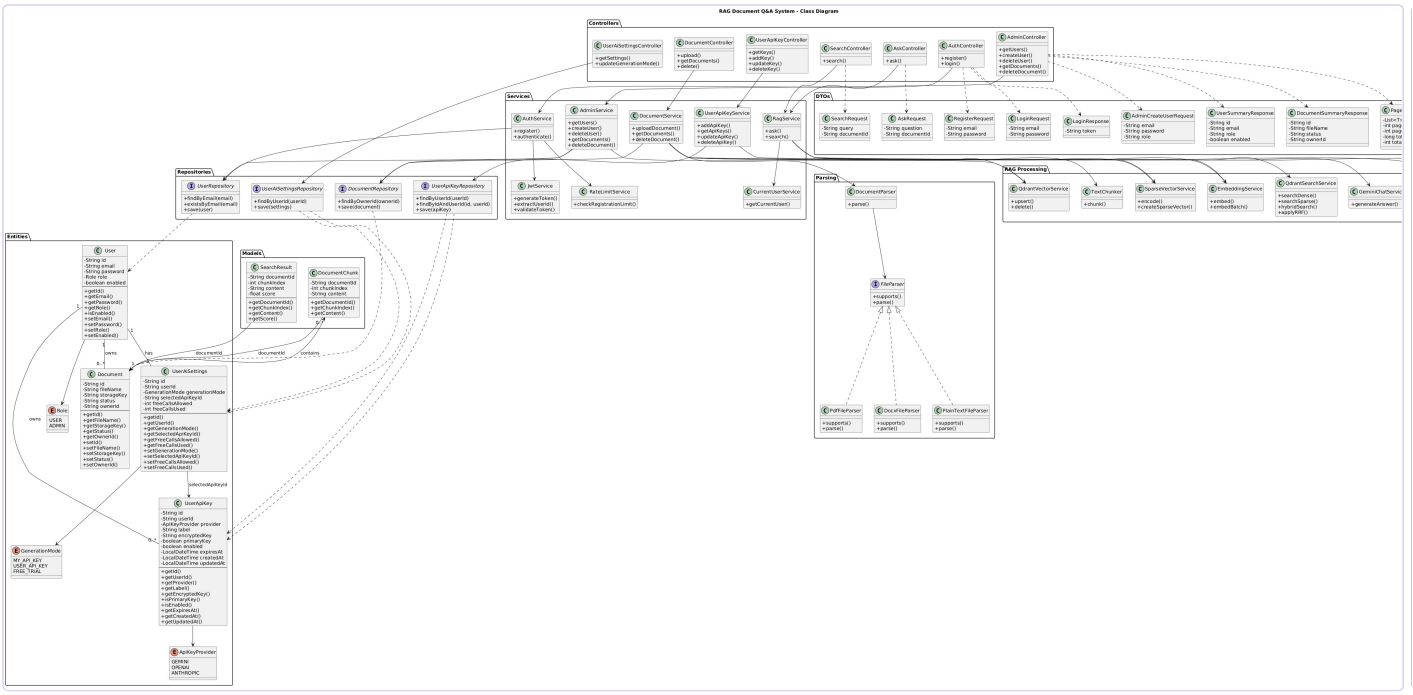
Use Case Diagram

- Read what the reader should notice: ownership and authentication are include-relationships on nearly every use case, not an afterthought bolted on at the controller.
- Search and Ask Question both funnel through the same Tenant Isolation and Generate AI Answer use cases — one retrieval path, two entry points.

06

Class Diagram

The backend follows a standard layered design: Controllers → Services → Repositories → Entities, with a separate Parsing hierarchy (a `FileParser` interface with PDF / DOCX / plain-text implementations) and a RAG Processing group (EmbeddingService, SparseVectorService, QdrantSearchService, QdrantVectorService, GeminiChatService).



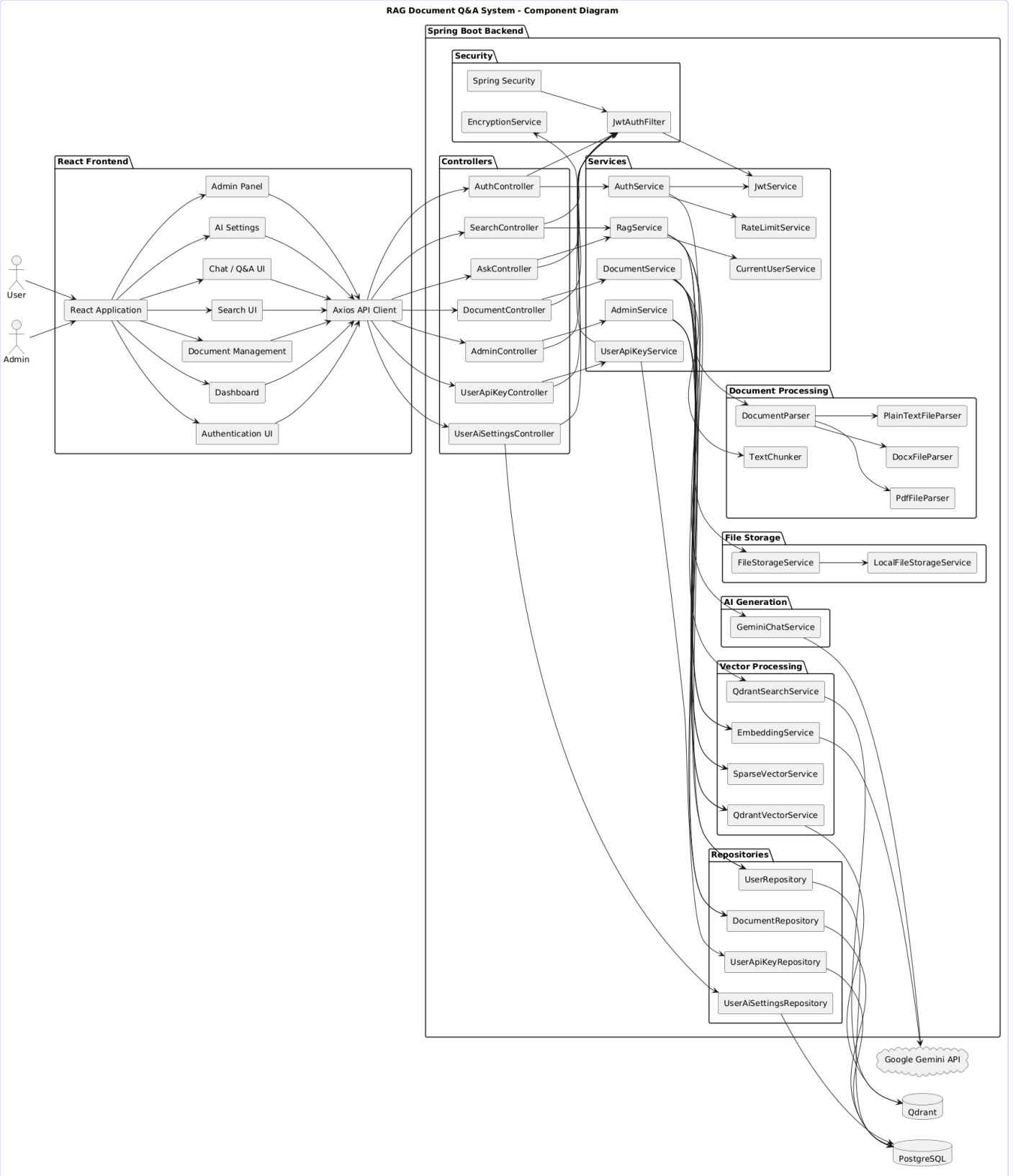
Class Diagram

- `User` owns `Document`, `UserAiSettings`, and `UserApiKey` — the three entities every tenant-isolation check pivots on.
- `FileParser` is a Strategy interface — adding a new file format means one new class, zero changes to `DocumentParser`.
- `GenerationMode` and `ApiKeyProvider` are modelled as enums, keeping the free-trial vs. BYOK decision type-safe end to end.

07

Component Diagram

This is the same architecture as the System Architecture diagram, one level deeper: every Spring Boot component is named, grouped by responsibility (Security, Controllers, Services, Document Processing, Vector Processing, Repositories), and wired to Qdrant, PostgreSQL, and the Gemini API.



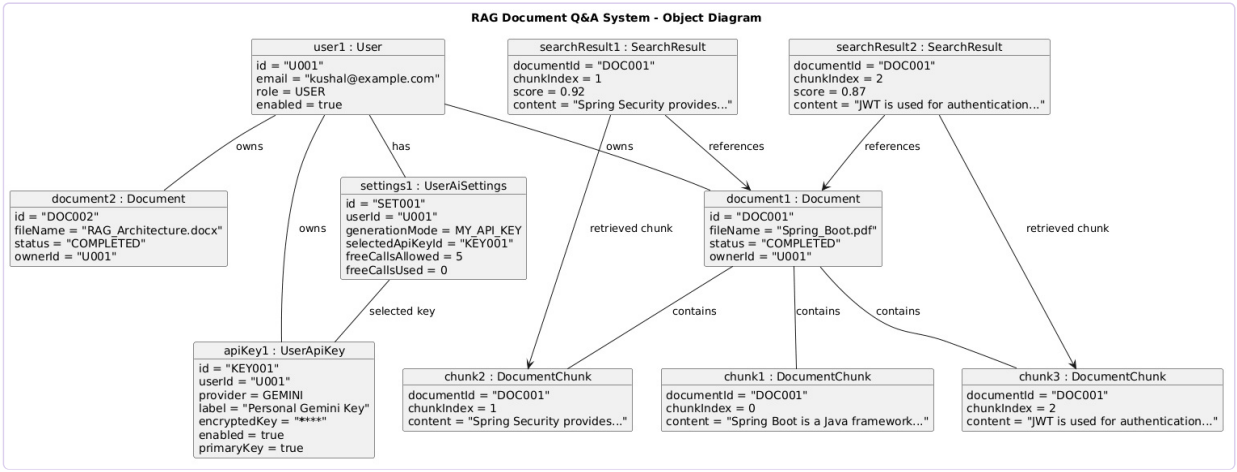
Component Diagram

- `JwtAuthFilter` sits in front of every controller — there is no path into the Services layer that skips authentication.
- Vector Processing is isolated into its own package (`QdrantSearchService` , `QdrantVectorService` , `EmbeddingService` , `SparseVectorService`), so the retrieval algorithm can change without touching controllers.

08

Object Diagram

A concrete snapshot of the model with real-shaped data: one user, two documents, an AI settings row pointing at a specific API key, and two ranked search results referencing their source chunks. This is the diagram that makes the abstract class model tangible.



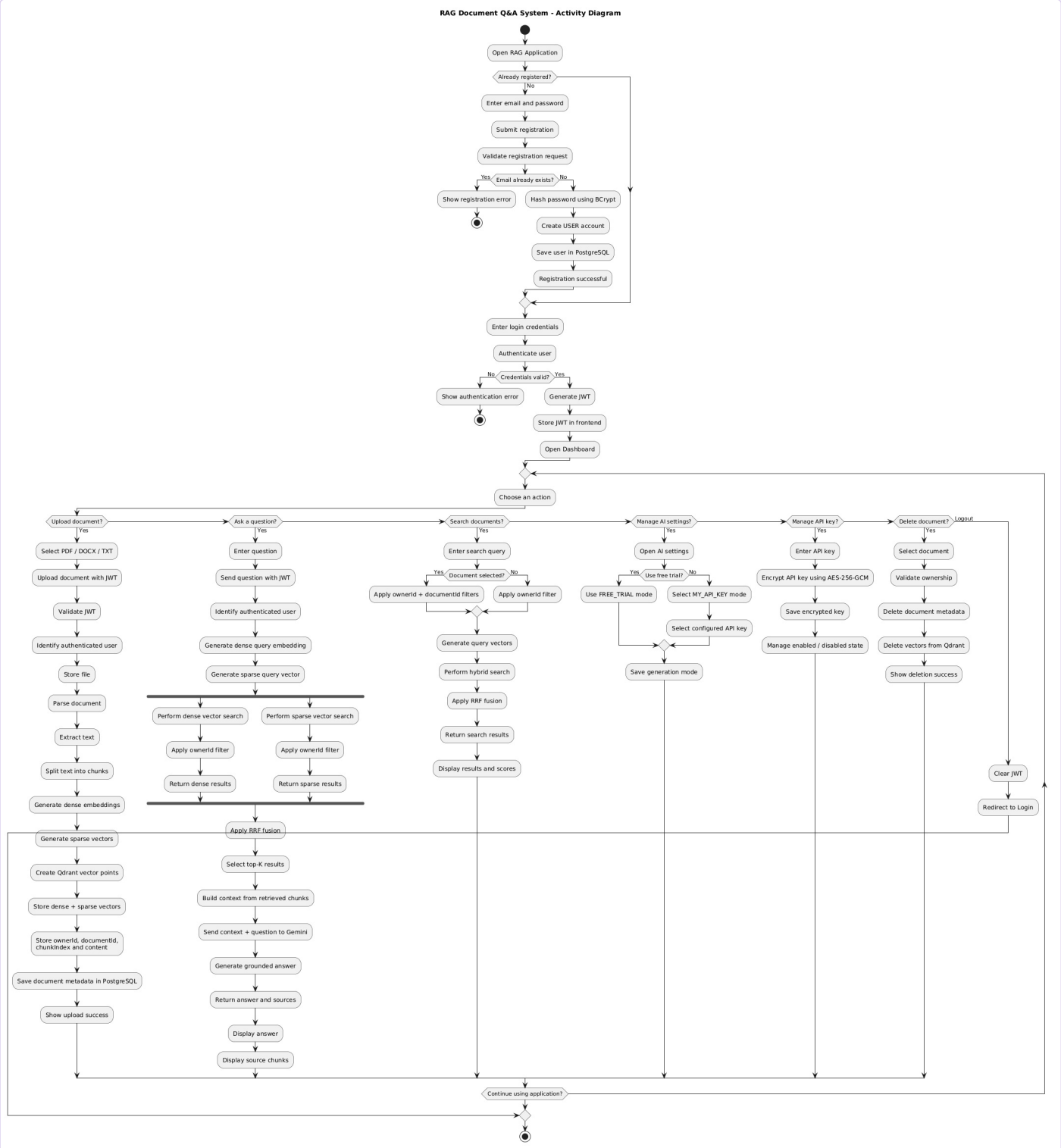
Object Diagram

- `searchResult1` and `searchResult2` both reference the same document but different chunks — this is what a single Ask Question call actually returns before an answer is generated.
- `settings1` shows the `MY_API_KEY` generation mode pointing at a specific `apiKey1` instance, not a hard-coded key.

09

Activity Diagram

The full user journey, branch by branch: registration, login, and then five parallel action paths from the dashboard — upload, ask, search, AI settings, and API key management — each modelled down to its Qdrant or PostgreSQL write.

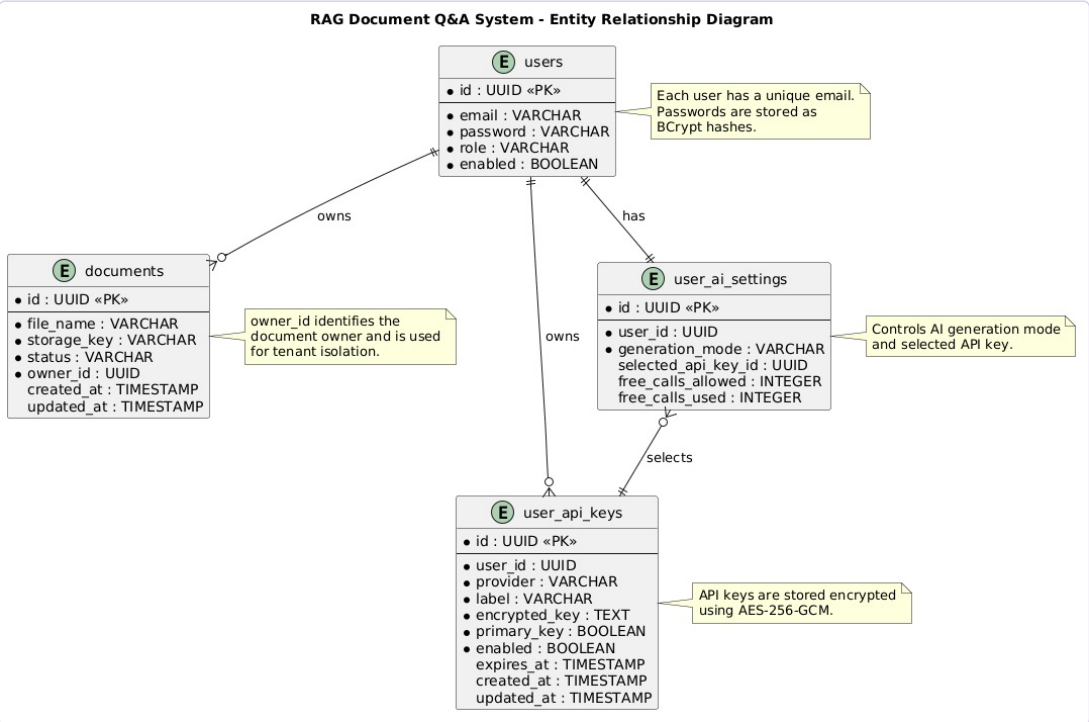


Activity Diagram

- Registration and login are separated into their own decision trees (email-exists check, credentials-valid check) before ever reaching the shared dashboard state.
- The Ask a Question branch shows the fork/join for parallel dense and sparse search explicitly, matching the actual `QdrantSearchService` implementation.

ER Diagram — PostgreSQL

The relational schema is deliberately small: four tables carry the entire application's structured state. Vector data lives entirely outside PostgreSQL, in Qdrant.

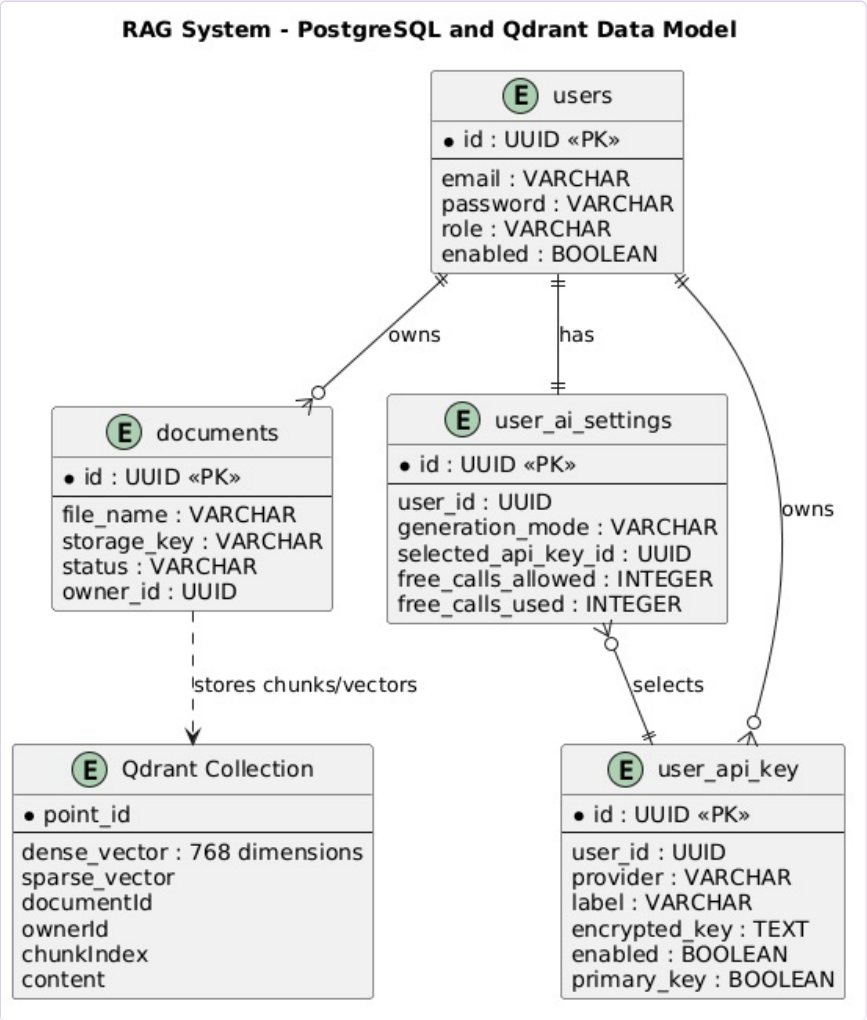


ER Diagram — PostgreSQL

- `owner_id` on `documents` is the tenant-isolation key referenced everywhere else in the system, including inside Qdrant's payload.
- Passwords are BCrypt hashes and API keys are AES-256-GCM ciphertext — nothing sensitive is stored in plain text.

ER Diagram — PostgreSQL + Qdrant

Qdrant isn't a relational store, so it doesn't get its own ERD entities in the usual sense — instead this diagram shows it as a linked collection that documents point into, carrying the same `ownerId` / `documentId` keys that PostgreSQL uses.

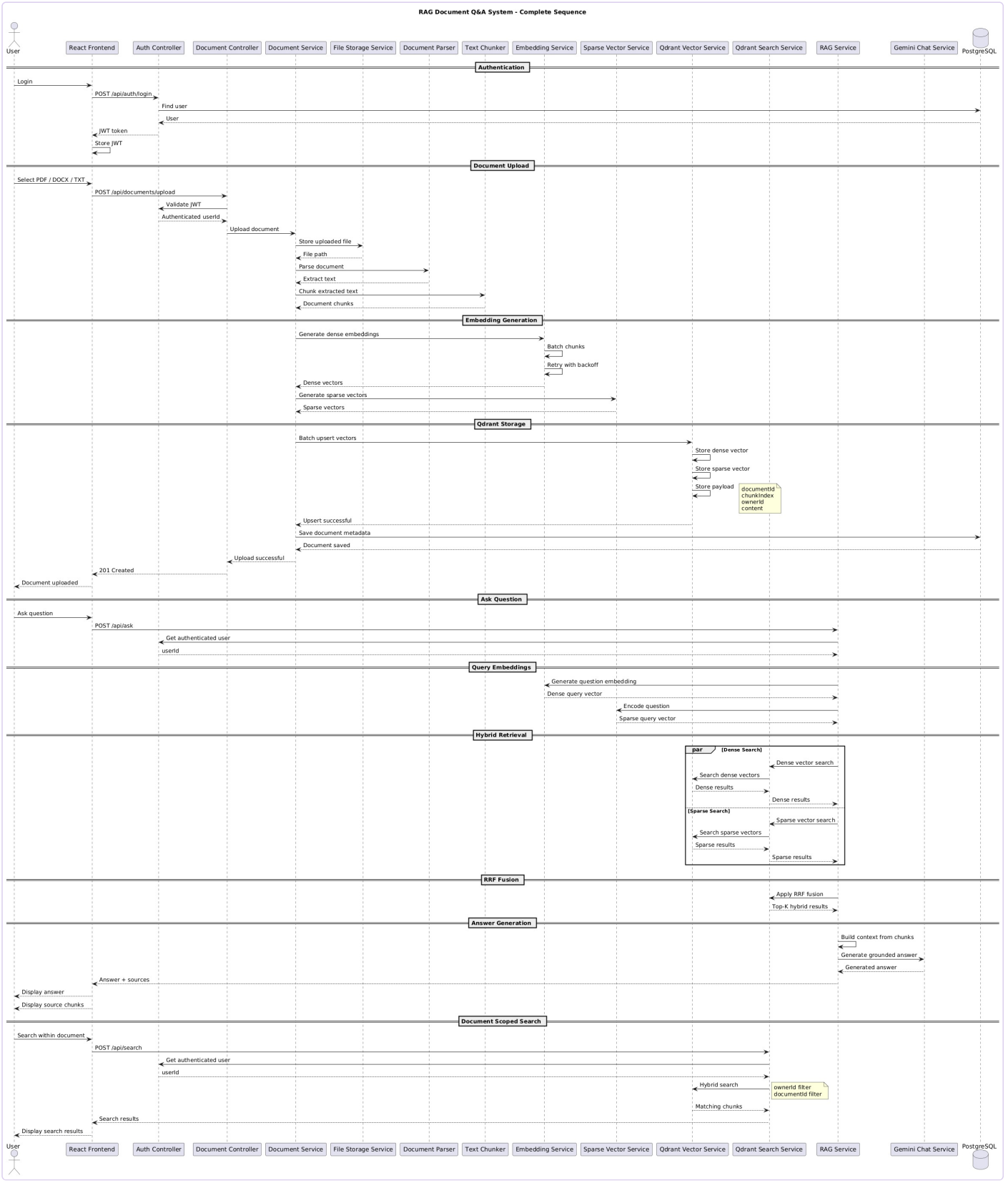


ER Diagram — PostgreSQL + Qdrant

- This is the diagram that makes the two-database design legible at a glance: relational metadata in PostgreSQL, semantic content in Qdrant, joined by shared identifiers rather than a foreign key.

Sequence Diagram — Complete Flow

The full request lifecycle across seven phases: Authentication, Document Upload, Embedding Generation, Qdrant Storage, Ask Question, Query Embeddings, Hybrid Retrieval (dense and sparse in parallel), RRF Fusion, Answer Generation, and Document-Scoped Search.

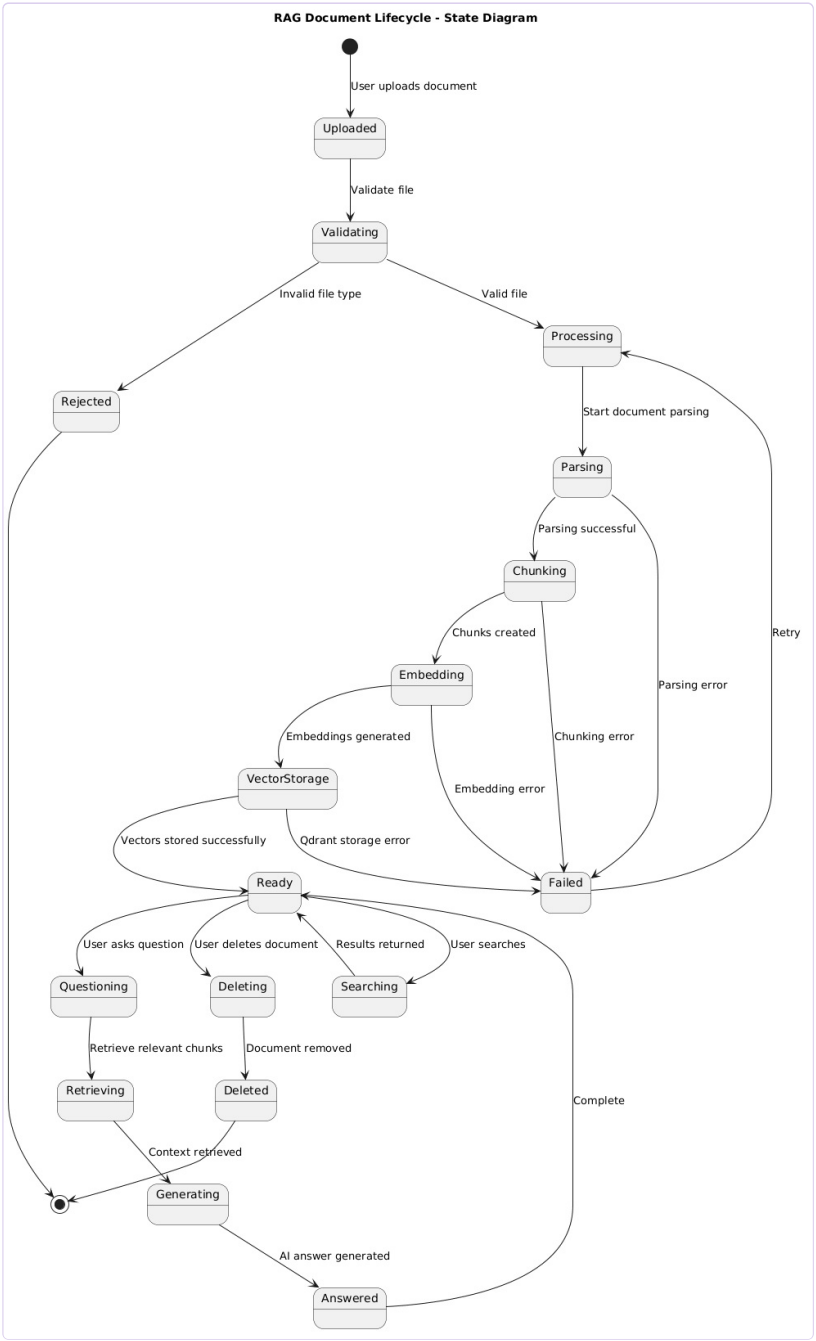


Sequence Diagram — Complete Flow

- The `par` fragment around Dense Search and Sparse Search is not decorative — `QdrantSearchService` genuinely issues both prefetch queries before fusing them.
- Embedding Generation shows batching and retry-with-backoff as first-class steps, because that's where the real quota problems on large documents showed up.

State Diagram — Document Lifecycle

Every uploaded document moves through a strict state machine: Uploaded → Validating → Processing → Parsing → Chunking → Embedding → VectorStorage → Ready, with a Failed state reachable (and retryable) from every processing step.



State Diagram — Document Lifecycle

- Once a document reaches `Ready`, it can be searched, questioned, or deleted — but never re-enters the ingestion states, keeping the lifecycle strictly forward-moving except for explicit retries.

Database & Vector Store

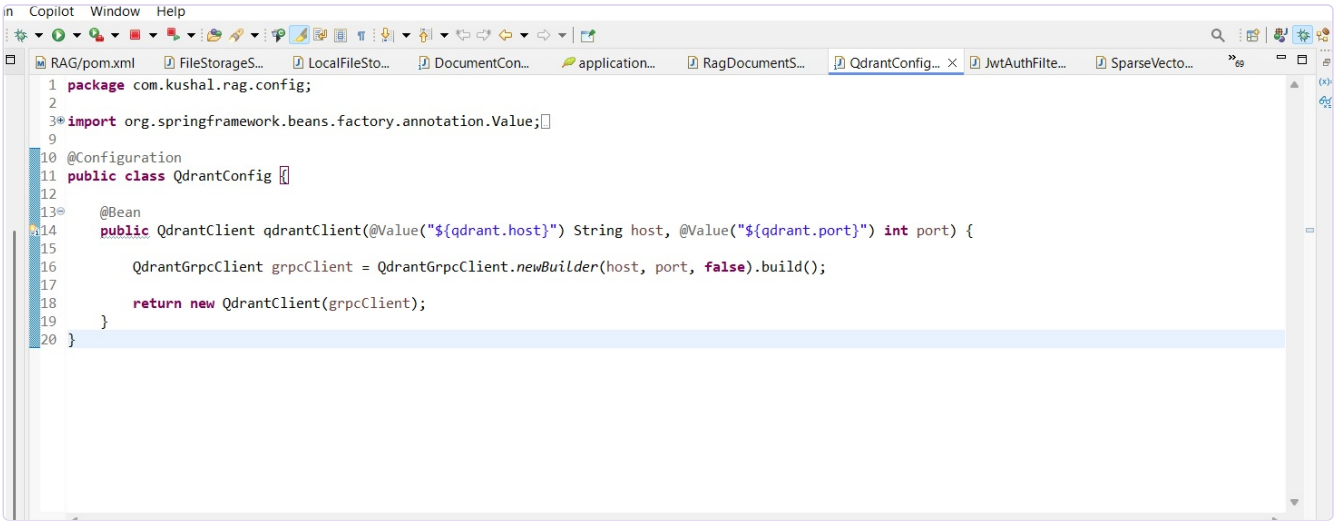
PostgreSQL and Qdrant carry very different kinds of data, and the design leans into that difference rather than trying to force one database to do both jobs.

PostgreSQL stores

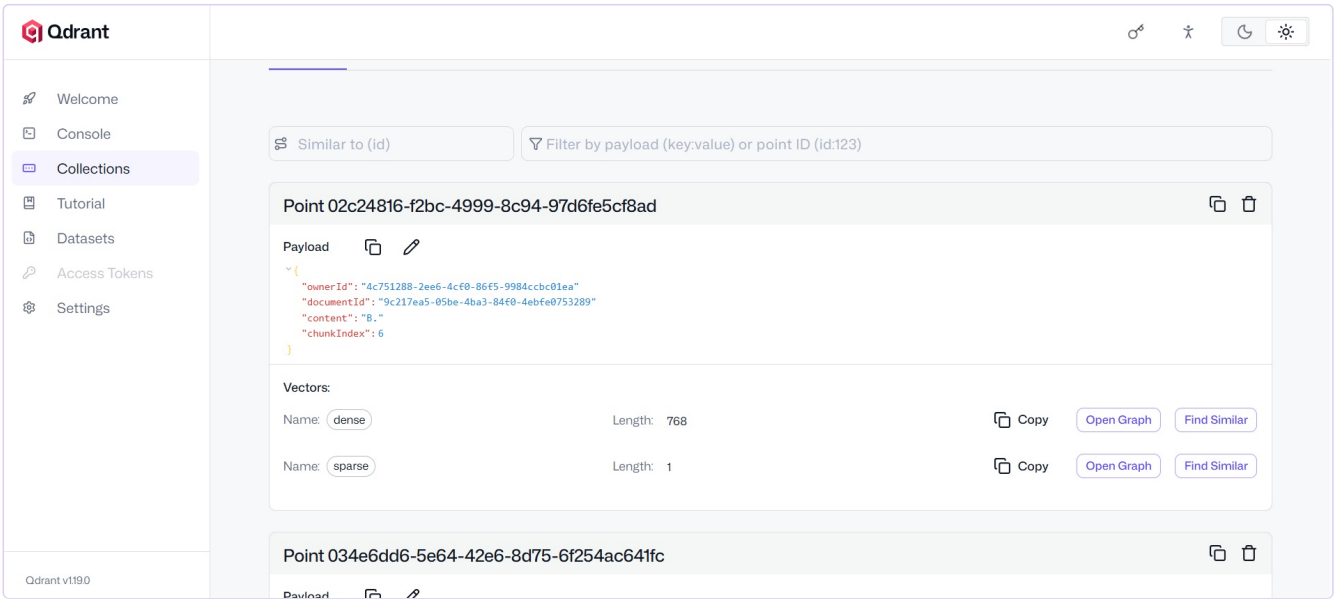
- Users, roles, credentials
- Document metadata and status
- AI generation settings per user
- Encrypted API keys

Qdrant stores

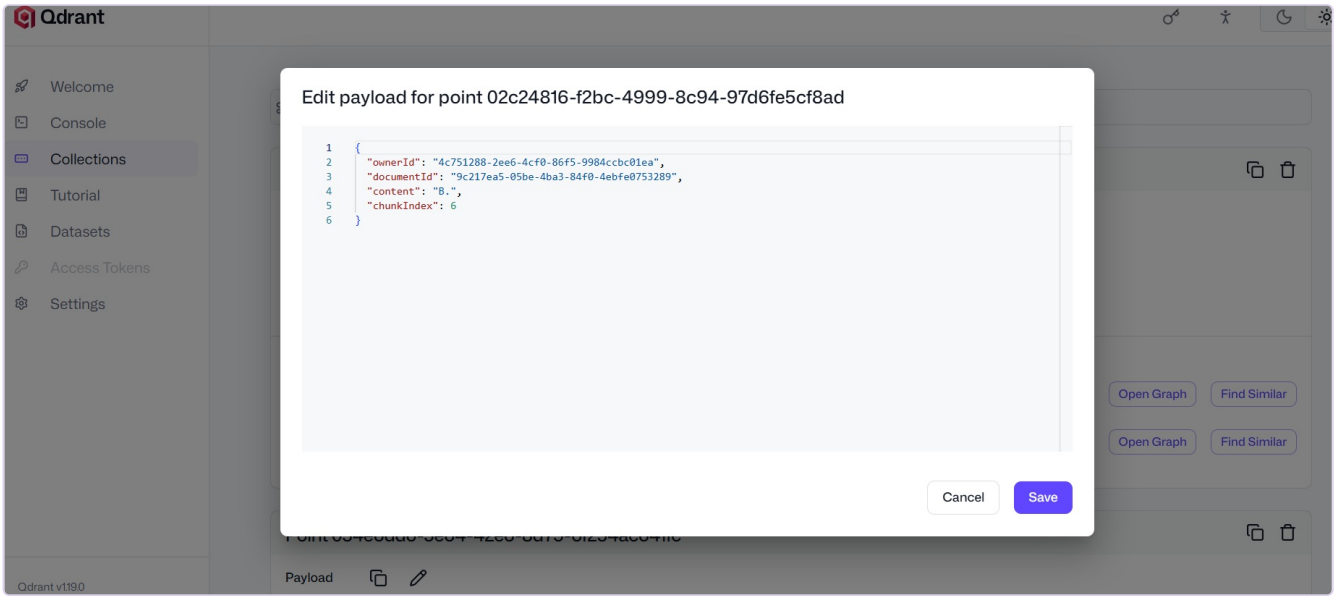
- Dense vectors (768 dimensions)
- Sparse vectors
- Chunk payload: documentId, ownerId, chunkIndex, content



QdrantConfig.java — QdrantClient bean wired over gRPC from host/port properties



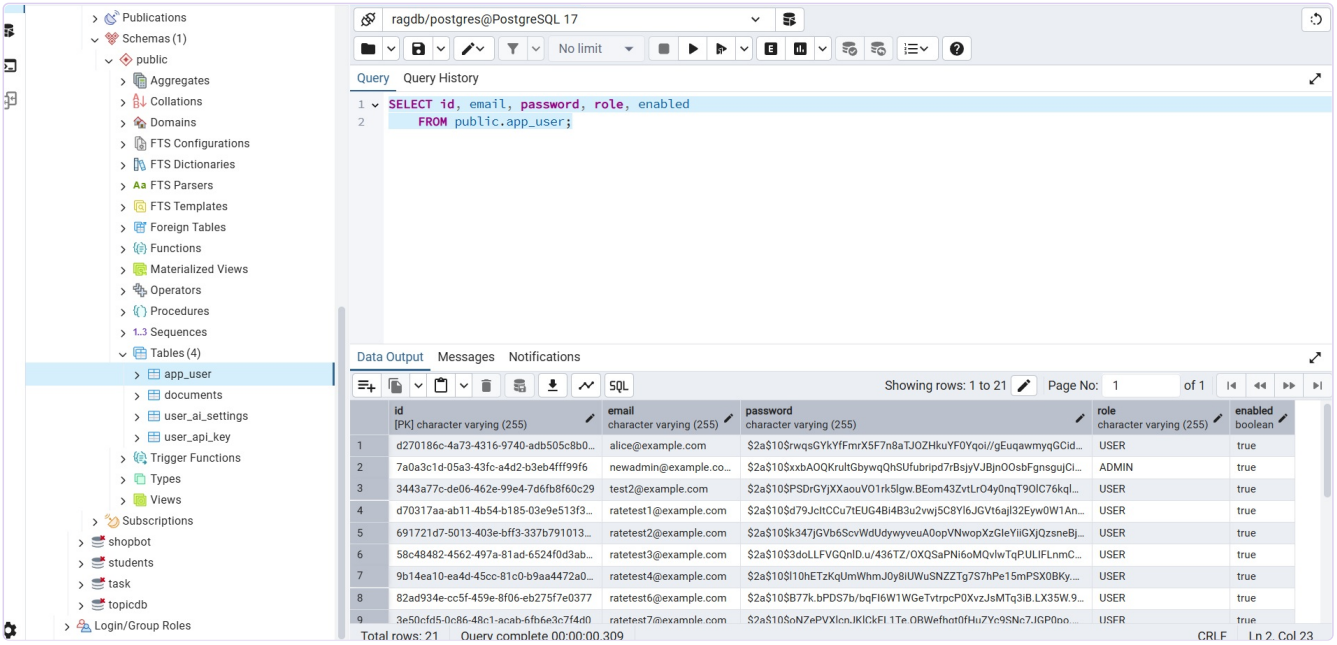
Qdrant Console — a stored point with its dense (768-dim) and sparse vectors, plus payload



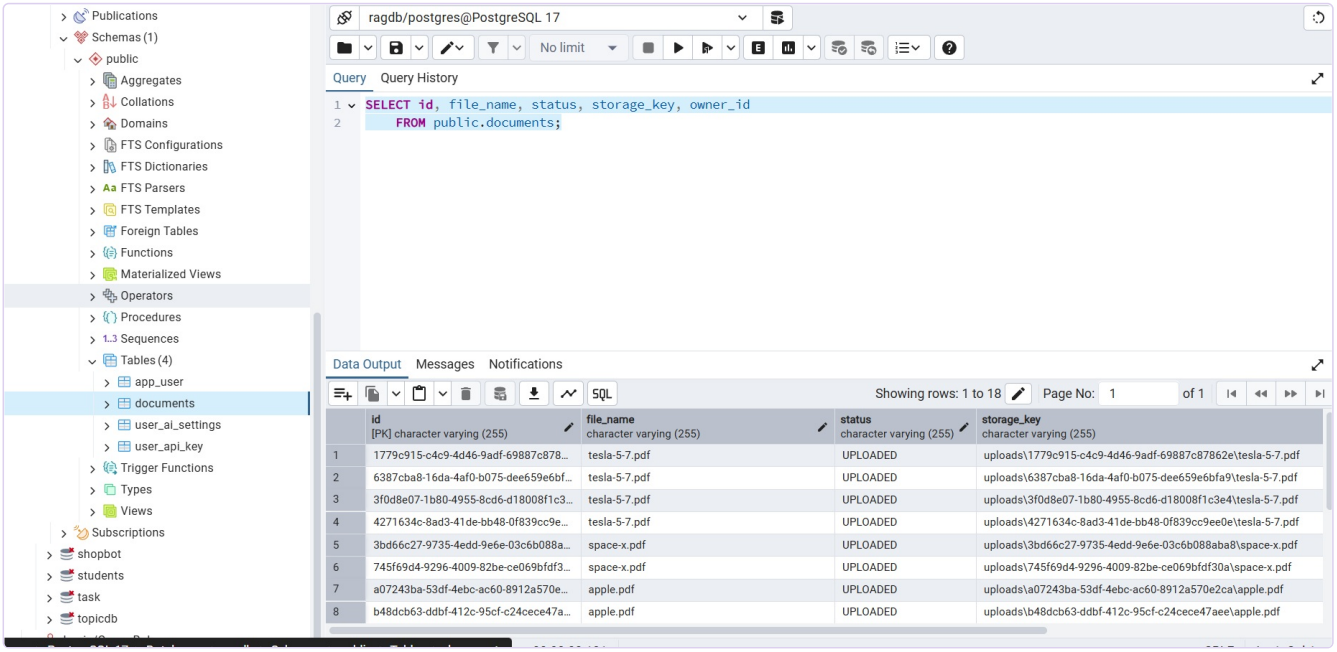
Editing a point's payload directly in the Qdrant console: ownerId, documentId, content, chunkIndex

Relational Data, Verified

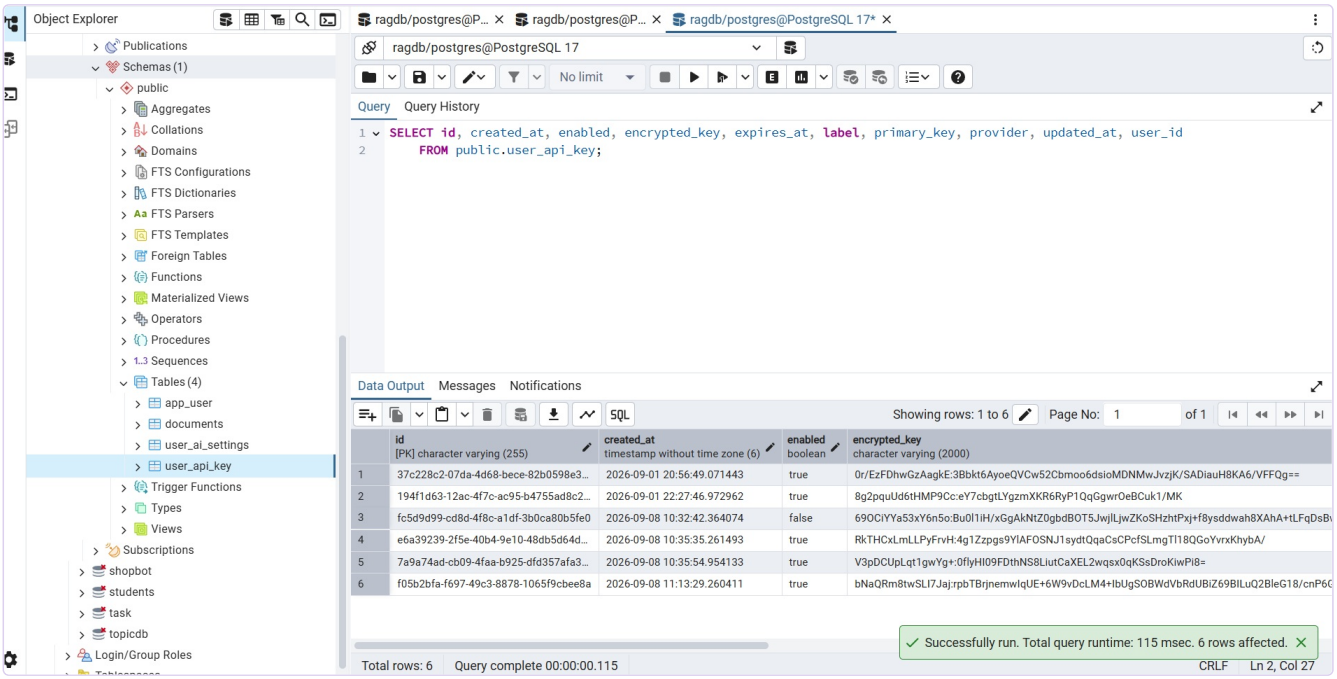
Beyond the diagrams, here is the actual PostgreSQL data behind the schema — queried directly against the running database.



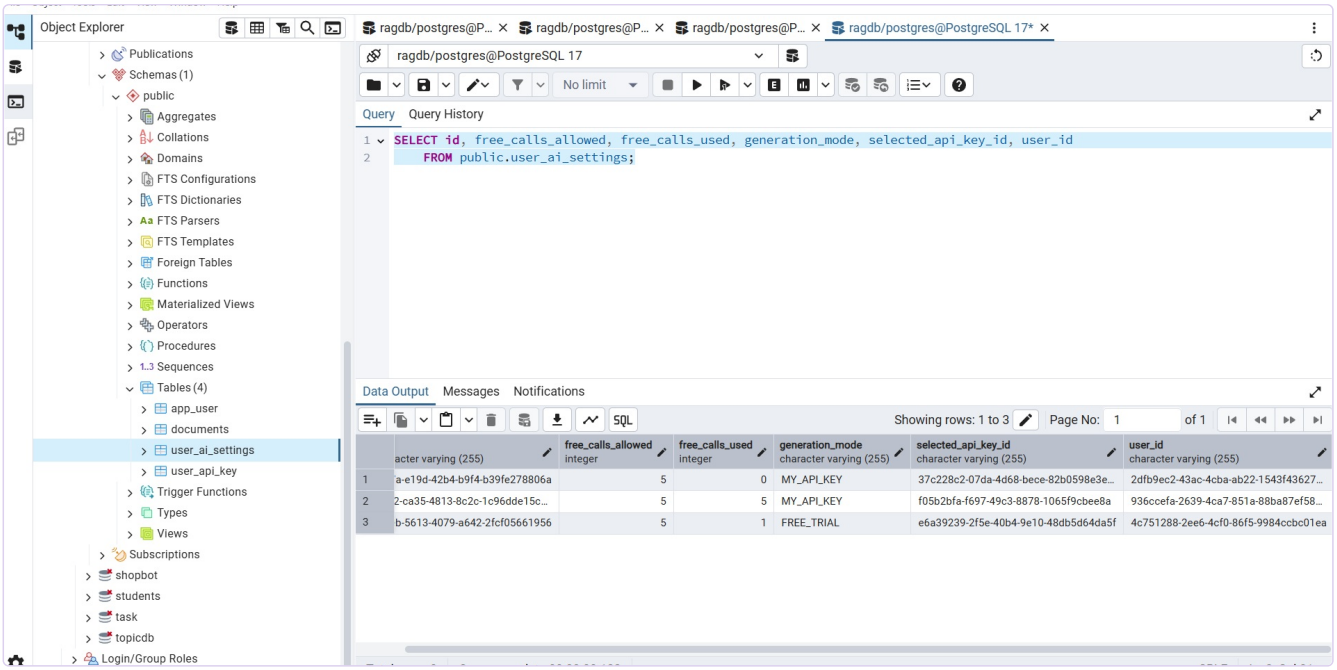
app_user table — BCrypt password hashes, roles, and enabled flags



documents table — file_name, status, storage_key and owner_id per upload



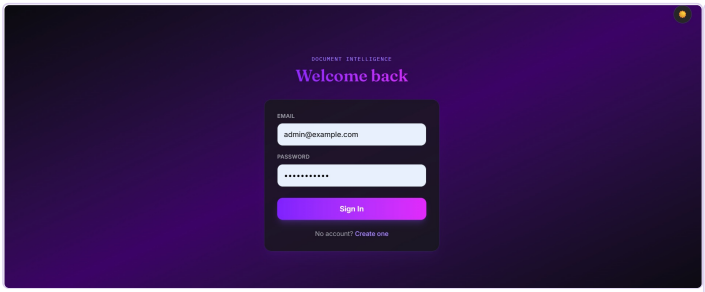
user_api_key table — AES-256-GCM encrypted_key values, never plaintext



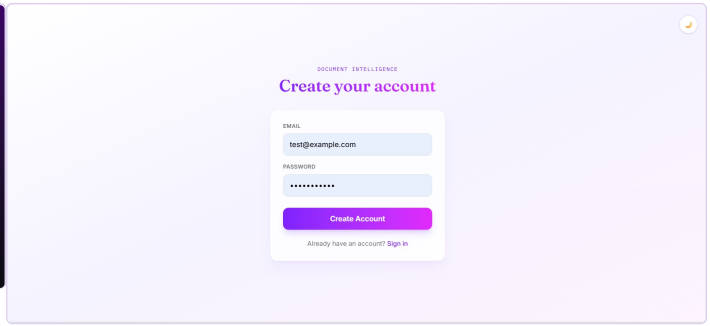
user_ai_settings table — per-user generation_mode, free-trial usage counters

Authentication & Security

Every request that touches user data passes through JWT authentication and owner-scoped filtering. Secrets never touch disk or the database in plain text.



Sign-in screen



Registration screen

```java
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {

    http

            .cors(cors -> cors.configurationSource(corsConfigurationSource()))

            .csrf(csrf -> csrf.disable())

            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))

            .authorizeHttpRequests(auth -> auth.requestMatchers("/api/auth/**").permitAll()
                    .requestMatchers("/api/admin/**").hasRole("ADMIN").anyRequest().authenticated())

            .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

    return http.build();
}

@Bean
public CorsConfigurationSource corsConfigurationSource() {

    CorsConfiguration configuration = new CorsConfiguration();

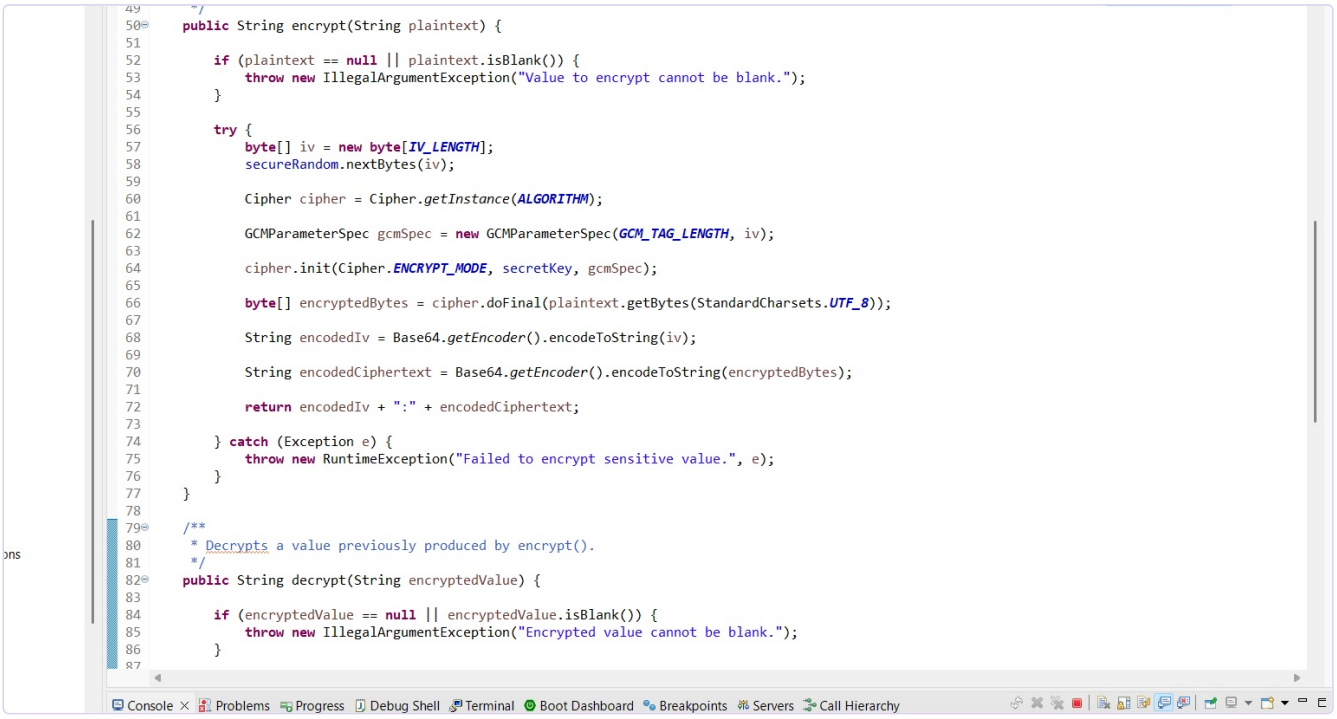
    // The frontend origins allowed to call this API.
    // Update this list once you know your actual frontend URL(s)
    // (e.g. "http://localhost:3000" for local dev,
    // "https://yourapp.com" once deployed).
    configuration.setAllowedOrigins(List.of("http://localhost:3000", "http://localhost:5173" // common Vite dev
                                                                                             // server port
    ));

    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
    configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
```

Console × Problems Progress Debug Shell Terminal Boot Dashboard Breakpoints Servers Call Hierarchy

*SecurityConfig.java — stateless sessions, CORS policy, role-gated /api/admin/**, JwtAuthFilter*



```java
    */
    public String encrypt(String plaintext) {

        if (plaintext == null || plaintext.isBlank()) {
            throw new IllegalArgumentException("Value to encrypt cannot be blank.");
        }

        try {
            byte[] iv = new byte[IV_LENGTH];
            secureRandom.nextBytes(iv);

            Cipher cipher = Cipher.getInstance(ALGORITHM);

            GCMParameterSpec gcmSpec = new GCMParameterSpec(GCM_TAG_LENGTH, iv);

            cipher.init(Cipher.ENCRYPT_MODE, secretKey, gcmSpec);

            byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));

            String encodedIv = Base64.getEncoder().encodeToString(iv);

            String encodedCiphertext = Base64.getEncoder().encodeToString(encryptedBytes);

            return encodedIv + ":" + encodedCiphertext;

        } catch (Exception e) {
            throw new RuntimeException("Failed to encrypt sensitive value.", e);
        }
    }

    /**
     * Decrypts a value previously produced by encrypt().
     */
    public String decrypt(String encryptedValue) {

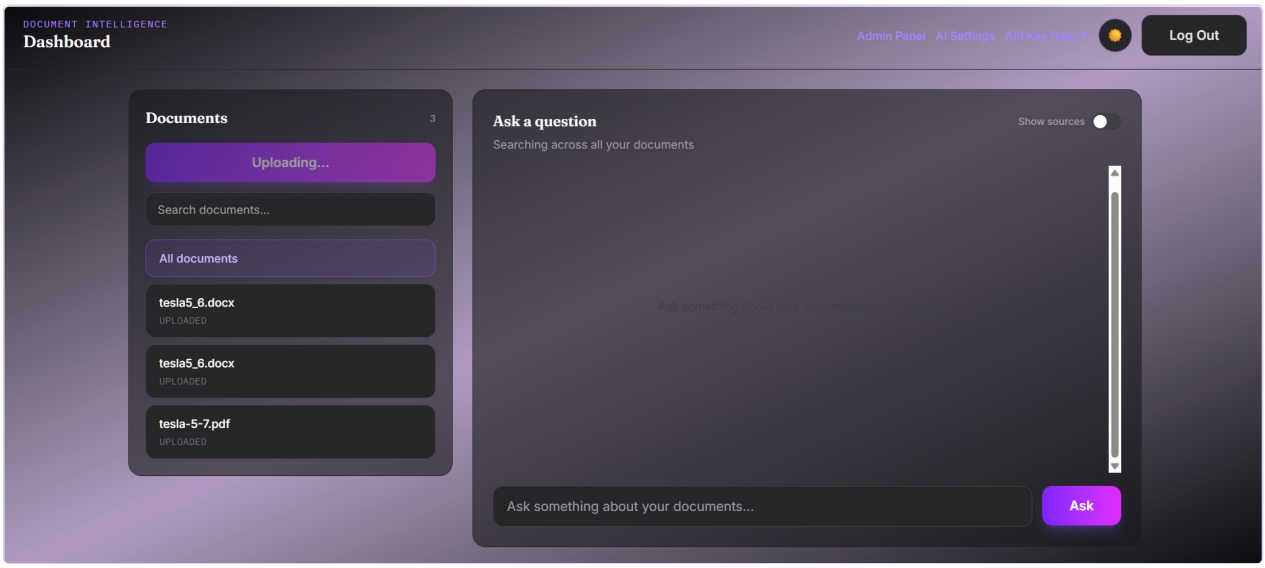
        if (encryptedValue == null || encryptedValue.isBlank()) {
            throw new IllegalArgumentException("Encrypted value cannot be blank.");
        }
```


EncryptionService.java — AES-256-GCM encrypt/decrypt with a random IV per value

- Passwords are hashed with BCrypt before they reach PostgreSQL.
- Sessions are stateless; every request re-validates its JWT via `JwtAuthFilter`.
- User-supplied Gemini API keys are encrypted at rest with AES-256-GCM and a per-value random IV.
- CSRF is disabled by design (stateless JWT API), CORS is scoped to explicit frontend origins.

Document Processing

Upload triggers a synchronous parse-then-async-embed pipeline. The parser is a Strategy pattern so new file types are a one-class addition, and chunking adapts its target size to document length.



Upload in progress from the dashboard

```java
package com.kushal.rag.service.parsing;

import java.io.IOException;

@Service
public class DocumentParser {

    private final List<FileParser> parsers;

    public DocumentParser(List<FileParser> parsers) {
        // Spring auto-injects every FileParser @Component bean here —
        // PdfFileParser, PlainTextFileParser, DocxFileParser, and any
        // future format we add, with zero changes needed in this class.
        this.parsers = parsers;
    }

    public String parse(String filename, InputStream inputStream) throws IOException {

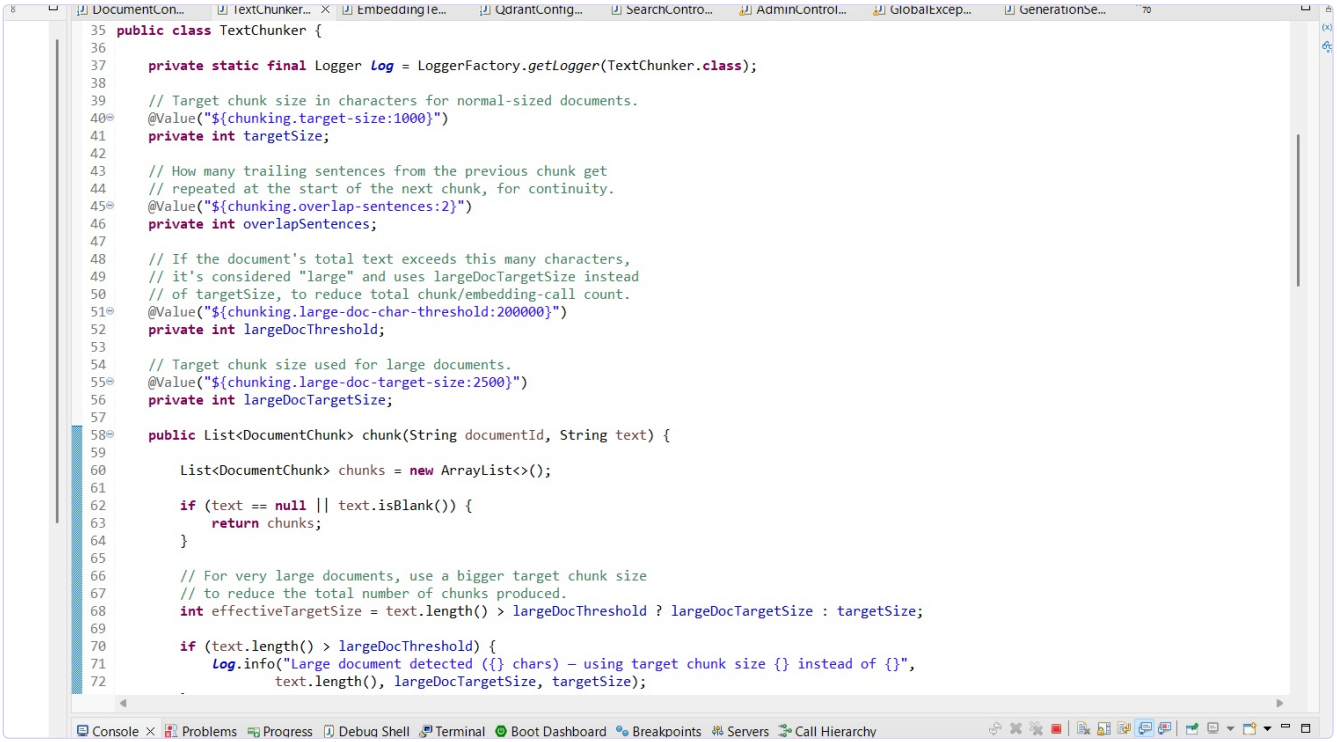
        String extension = extractExtension(filename);

        FileParser parser = parsers.stream()
                .filter(p -> p.supports(extension))
                .findFirst()
                .orElseThrow(() -> new UnsupportedFileTypeException(
                        "Unsupported file type: ." + extension
                                + ". Supported types: pdf, docx, txt, md"));

        return parser.parse(inputStream);
    }

    private String extractExtension(String filename) {
        if (filename == null || !filename.contains(".")) {
            throw new UnsupportedFileTypeException("File has no extension: " + filename);
        }
        return filename.substring(filename.lastIndexOf('.') + 1).toLowerCase();
    }
}
```

DocumentParser.java — dispatches to the right FileParser by extension via Spring auto-injection



```java
public class TextChunker {

    private static final Logger log = LoggerFactory.getLogger(TextChunker.class);

    // Target chunk size in characters for normal-sized documents.
    @Value("${chunking.target-size:1000}")
    private int targetSize;

    // How many trailing sentences from the previous chunk get
    // repeated at the start of the next chunk, for continuity.
    @Value("${chunking.overlap-sentences:2}")
    private int overlapSentences;

    // If the document's total text exceeds this many characters,
    // it's considered "large" and uses largeDocTargetSize instead
    // of targetSize, to reduce total chunk/embedding-call count.
    @Value("${chunking.large-doc-char-threshold:200000}")
    private int largeDocThreshold;

    // Target chunk size used for large documents.
    @Value("${chunking.large-doc-target-size:2500}")
    private int largeDocTargetSize;

    public List<DocumentChunk> chunk(String documentId, String text) {

        List<DocumentChunk> chunks = new ArrayList<>();

        if (text == null || text.isBlank()) {
            return chunks;
        }

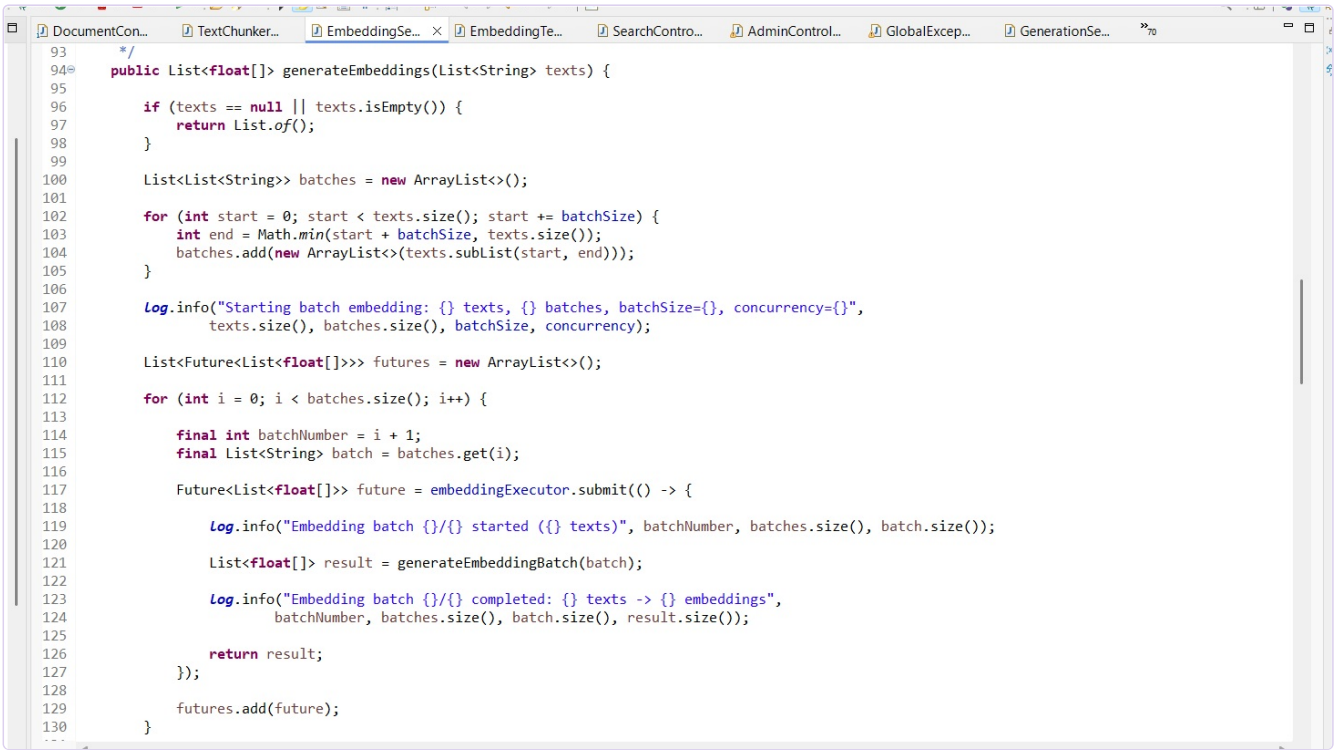
        // For very large documents, use a bigger target chunk size
        // to reduce the total number of chunks produced.
        int effectiveTargetSize = text.length() > largeDocThreshold ? largeDocTargetSize : targetSize;

        if (text.length() > largeDocThreshold) {
            log.info("Large document detected ({} chars) — using target chunk size {} instead of {}",
                    text.length(), largeDocTargetSize, targetSize);
```

TextChunker.java — configurable target size, sentence overlap, and adaptive sizing for large documents

> A document over the configured character threshold automatically switches to a larger target chunk size, trading a little retrieval granularity for far fewer embedding calls on very large files.

Hybrid Search & RAG



```java
    */
    public List<float[]> generateEmbeddings(List<String> texts) {

        if (texts == null || texts.isEmpty()) {
            return List.of();
        }

        List<List<String>> batches = new ArrayList<>();

        for (int start = 0; start < texts.size(); start += batchSize) {
            int end = Math.min(start + batchSize, texts.size());
            batches.add(new ArrayList<>(texts.subList(start, end)));
        }

        log.info("Starting batch embedding: {} texts, {} batches, batchSize={}, concurrency={}",
                texts.size(), batches.size(), batchSize, concurrency);

        List<Future<List<float[]>>> futures = new ArrayList<>();

        for (int i = 0; i < batches.size(); i++) {

            final int batchNumber = i + 1;
            final List<String> batch = batches.get(i);

            Future<List<float[]>> future = embeddingExecutor.submit(() -> {

                log.info("Embedding batch {}/{} started ({} texts)", batchNumber, batches.size(), batch.size());

                List<float[]> result = generateEmbeddingBatch(batch);

                log.info("Embedding batch {}/{} completed: {} texts -> {} embeddings",
                        batchNumber, batches.size(), batch.size(), result.size());

                return result;
            });

            futures.add(future);
        }
```

EmbeddingService.java — batched, async embedding generation with per-batch logging



```java
        // 3. Build the shared filter — ownerId always required,
        // documentId optional.
        Filter.Builder filterBuilder = Filter.newBuilder().addMust(ConditionFactory.matchKeyword("ownerId", ownerId));

        if (documentId != null && !documentId.isBlank()) {
            filterBuilder.addMust(ConditionFactory.matchKeyword("documentId", documentId));
        }

        Filter filter = filterBuilder.build();

        // 4. Prefetch #1 — dense (semantic) search.
        PrefetchQuery densePrefetch = PrefetchQuery.newBuilder()
                .setQuery(Query.newBuilder().setNearest(VectorInput.newBuilder().setDense(denseVector).build()).build())
                .setUsing("dense").setFilter(filter).setLimit(limit * 2L).build();

        // 5. Prefetch #2 — sparse (exact keyword) search.
        PrefetchQuery sparsePrefetch = PrefetchQuery.newBuilder()
                .setQuery(
                        Query.newBuilder().setNearest(VectorInput.newBuilder().setSparse(sparseVector).build()).build())
                .setUsing("sparse").setFilter(filter).setLimit(limit * 2L).build();

        // 6. Final fused query using Reciprocal Rank Fusion (RRF).
        QueryPoints queryRequest = QueryPoints.newBuilder().setCollectionName(collectionName).addPrefetch(densePrefetch)
                .addPrefetch(sparsePrefetch).setQuery(Query.newBuilder().setFusion(Fusion.RRF).build()).setLimit(limit)
                .setWithPayload(WithPayloadSelectorFactory.enable(true)).build();

        try {

            List<ScoredPoint> scoredPoints = qdrantClient.queryAsync(queryRequest).get();

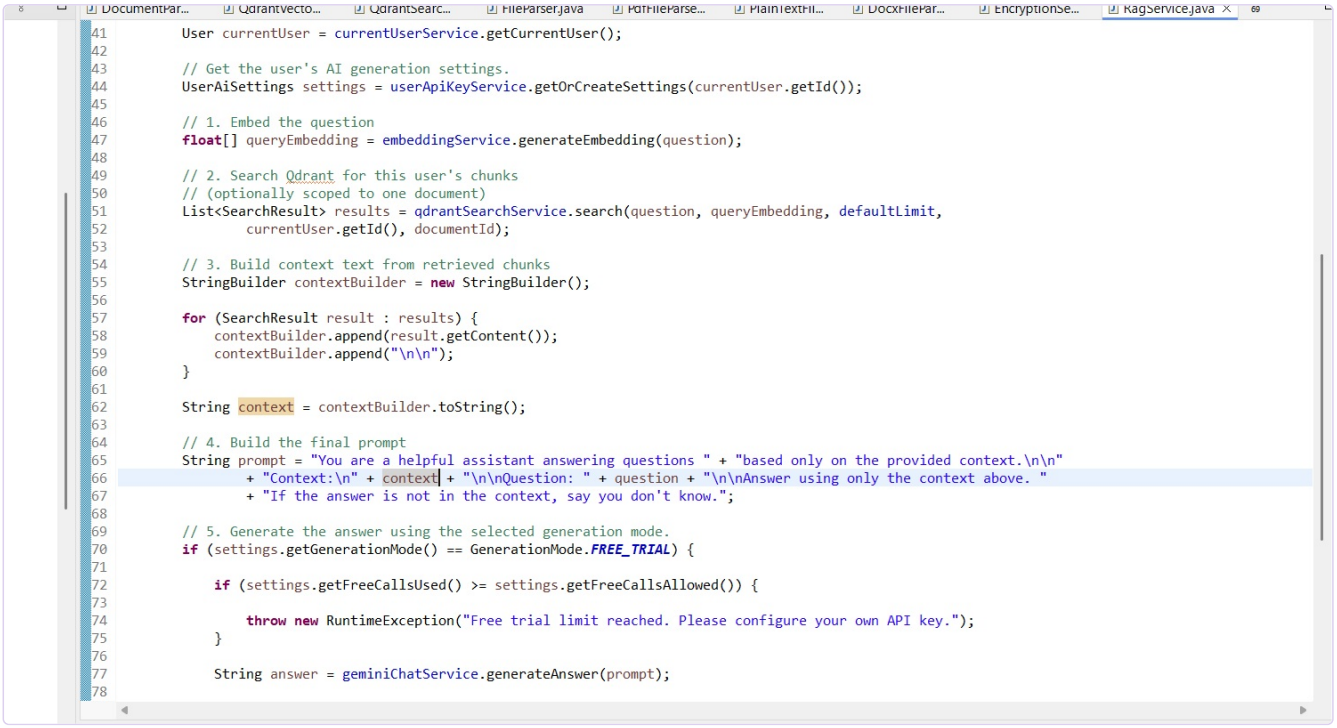
            List<SearchResult> results = new ArrayList<>();

            for (ScoredPoint point : scoredPoints) {

                Map<String, Value> payload = point.getPayloadMap();

                String resultDocumentId = payload.containsKey("documentId") ? payload.get("documentId").getStringValue()
                        : "unknown";
```

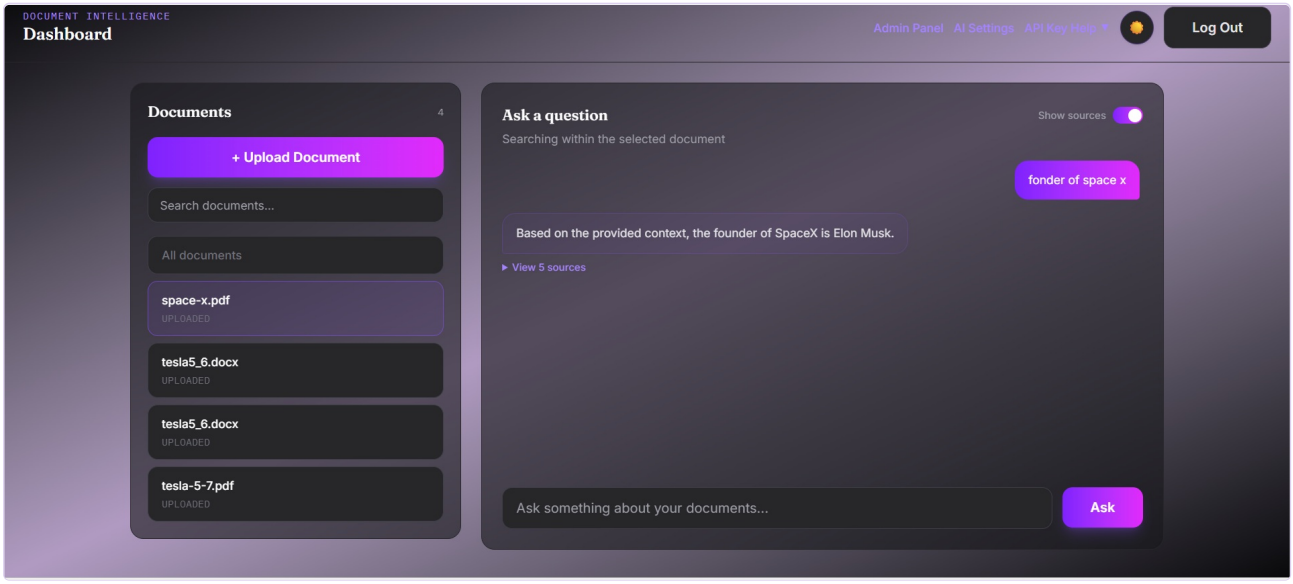
QdrantSearchService.java — dense + sparse prefetch queries fused with RRF, owner/document filtering



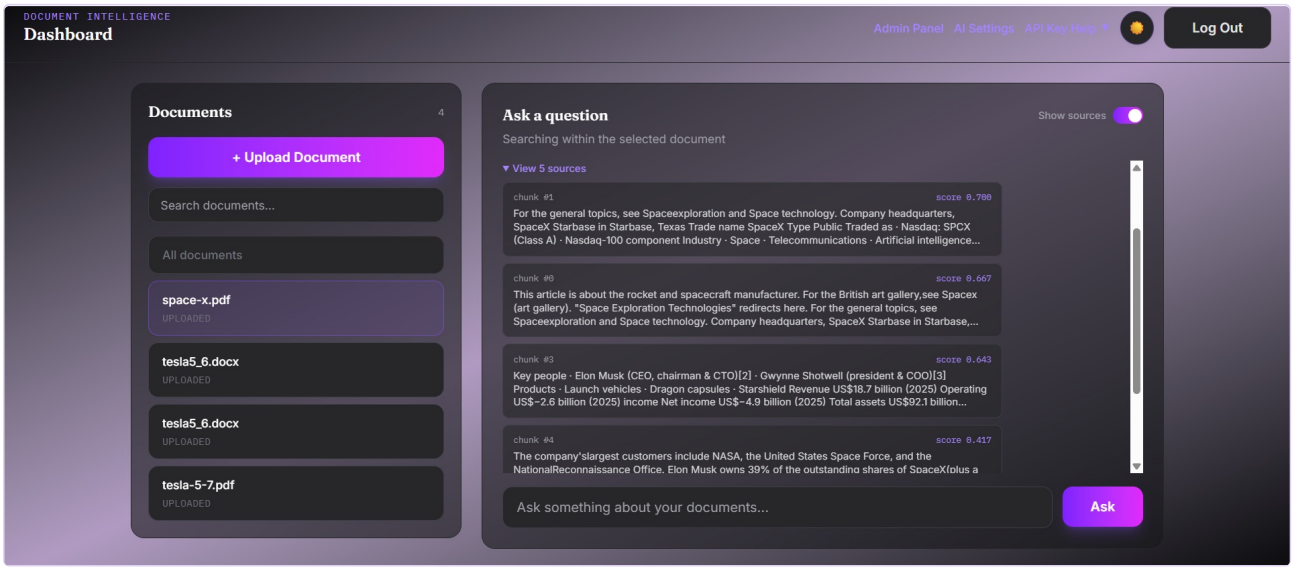
RagService.java — embed question, retrieve scoped chunks, build prompt, enforce free-trial quota, generate answer

Ask a Question, See the Sources

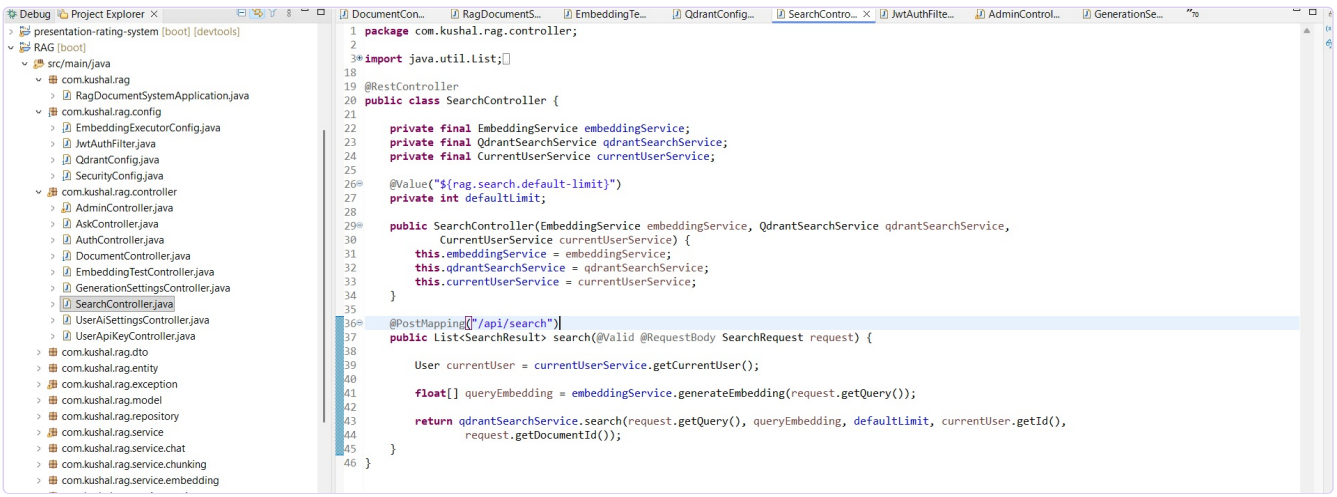
Every answer is grounded and auditable: the user can expand the exact chunks the model was given, each with its retrieval score.



A grounded answer, scoped to a single document, with a collapsible source list



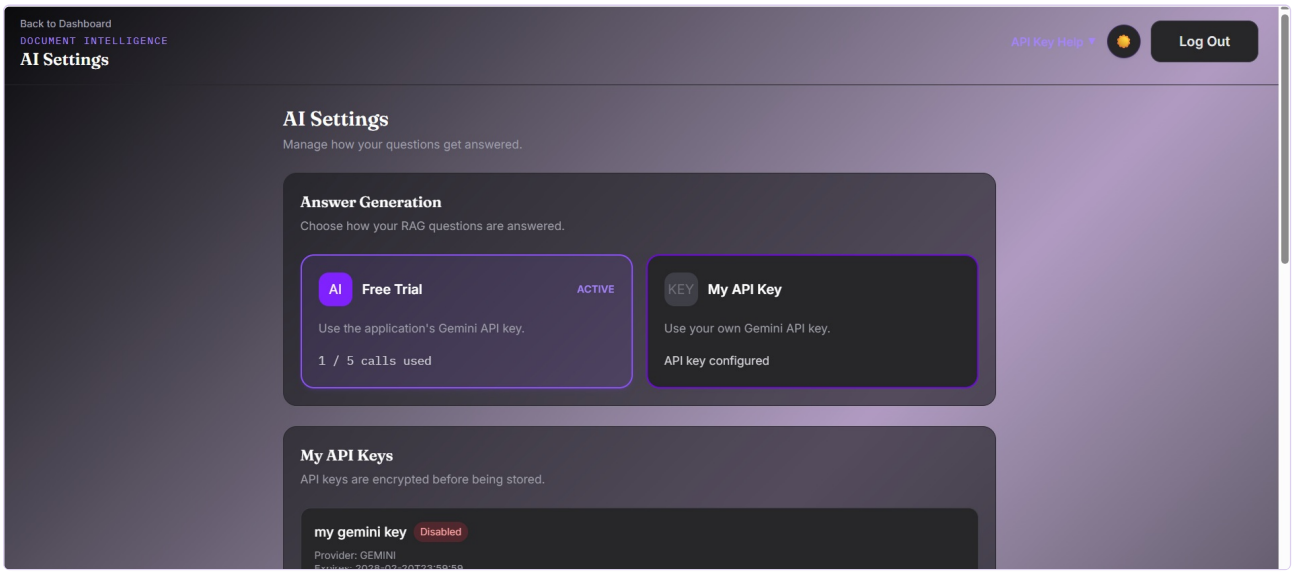
Expanded sources — individual chunks with hybrid retrieval scores



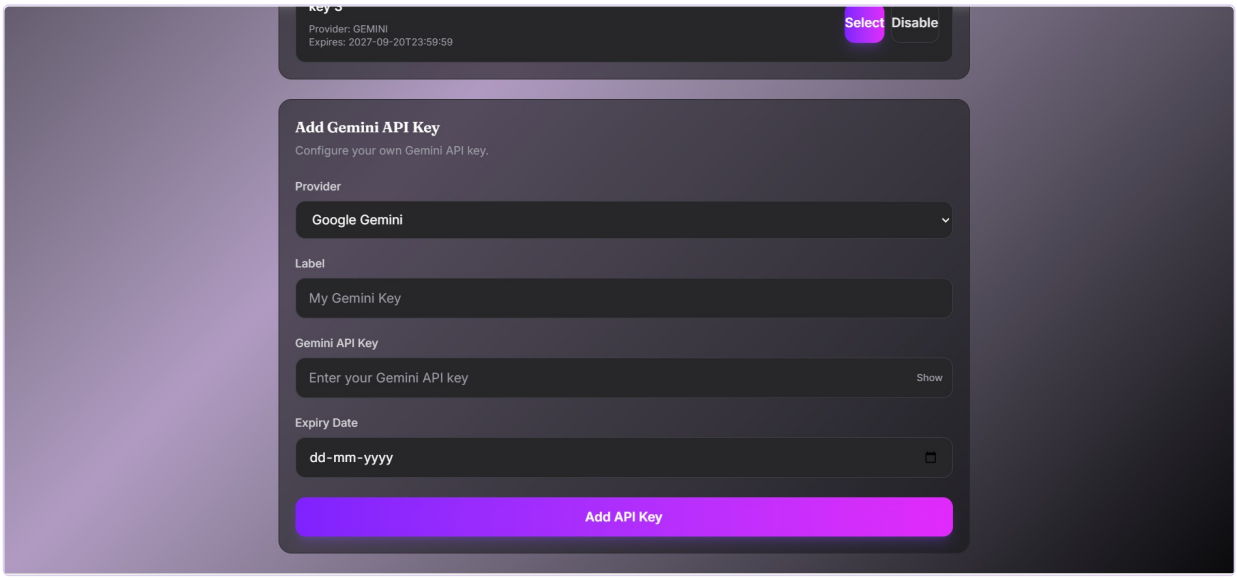
SearchController.java — embeds the query and delegates to QdrantSearchService with the authenticated user's ID

BYOK & AI Settings

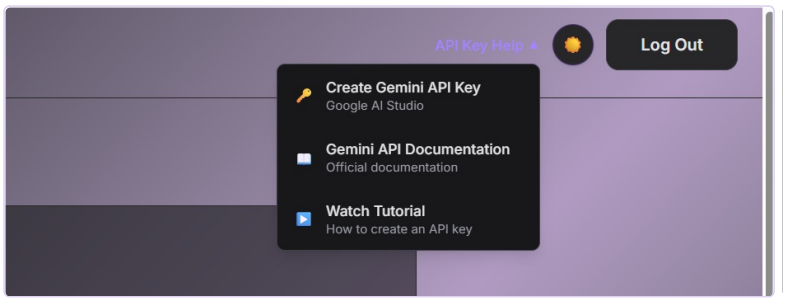
New users get a limited free trial on the application's own Gemini key; anyone can switch to their own key for unlimited use. Keys are entered once, encrypted immediately, and never displayed again in plain text.



AI Settings — Free Trial (usage counter) vs. My API Key, plus a list of configured keys



Adding a Gemini API key — provider, label, key, optional expiry

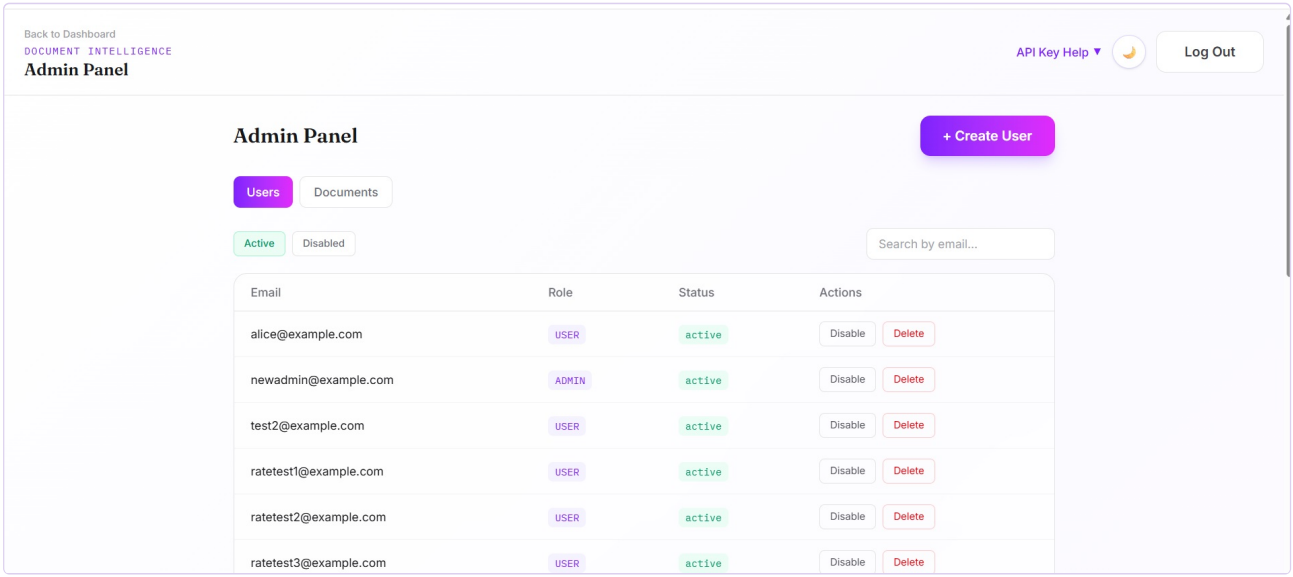


In-app help for users who don't yet have a Gemini key

> **Why BYOK only applies to generation, not embeddings:** vectors already stored in Qdrant were generated with one embedding model. Letting a user swap embedding providers mid-project would make their existing vectors incompatible with new queries, silently breaking retrieval.

Admin Panel

Admins get a dedicated view over users and documents system-wide, independent of tenant isolation, for moderation and support.



Admin Panel — user list with role, status, and disable/delete actions

```java
            @RequestParam(required = false) Integer size) {

        int effectiveSize = (size != null) ? size : defaultPageSize;

        return adminService.listAllUsers(page, effectiveSize);
    }

    @GetMapping("/api/admin/documents")
    public PagedResponse<DocumentSummaryResponse> listAllDocuments(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(required = false) Integer size) {

        int effectiveSize = (size != null) ? size : defaultPageSize;

        return adminService.listAllDocuments(page, effectiveSize);
    }
    @PostMapping("/api/admin/createUser")
    public UserSummaryResponse createUser(@RequestBody AdminCreateUserRequest request) {

        return adminService.createUser(request.getEmail(), request.getPassword(), request.getRole());
    }

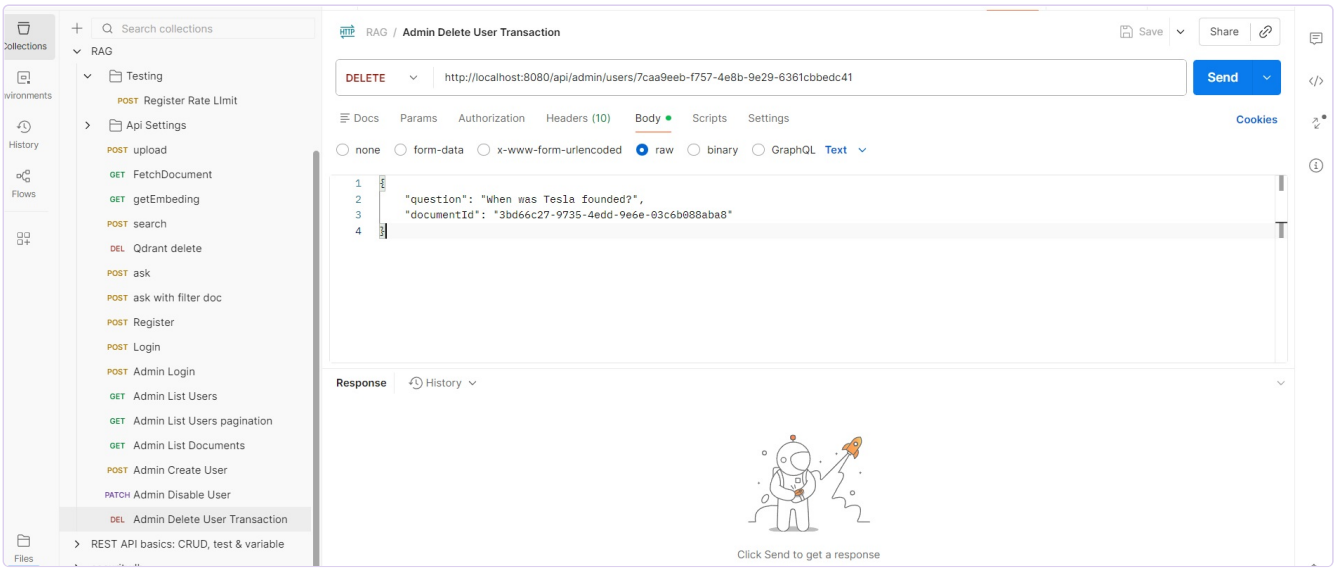
    @PatchMapping("/api/admin/users/{id}/disable")
    public UserSummaryResponse disableUser(@PathVariable String id) {
        return adminService.disableUser(id);
    }

    @DeleteMapping("/api/admin/users/{id}")
    public String hardDeleteUser(@PathVariable String id) {
        adminService.hardDeleteUser(id);
        return "User and all associated data deleted successfully.";
    }
    @PatchMapping("/api/admin/users/{id}/enable")
    public UserSummaryResponse enableUser(@PathVariable String id) {
        return adminService.enableUser(id);
    }
```

AdminController.java — paginated user/document listing, create/disable/enable/hard-delete

API Testing

Every endpoint was exercised through Postman during development — authentication, document-scoped questions, admin operations, and Qdrant cleanup on user deletion.

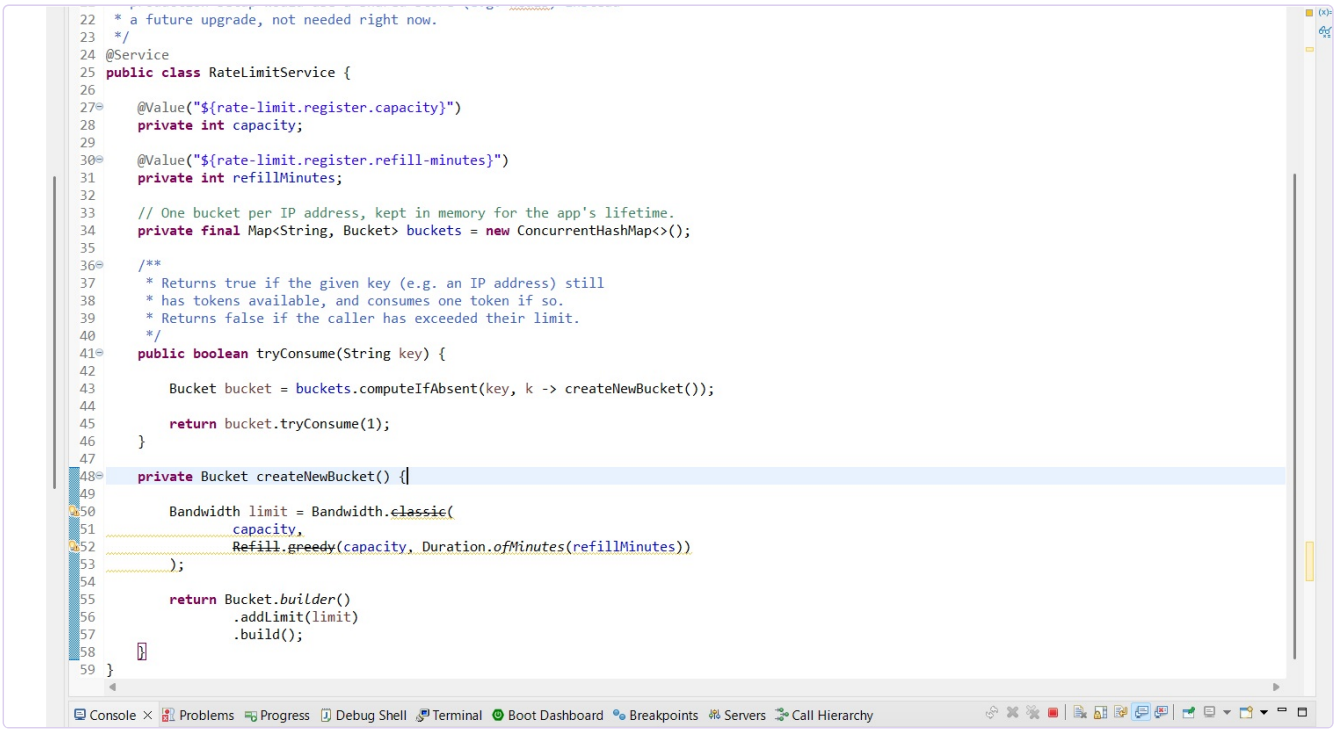


Postman collection — admin, auth, ask, search, and Qdrant-cleanup requests against the running API

Endpoint	Purpose
`POST /api/auth/register` / `/login`	Account creation and JWT issuance
`POST /api/documents/upload`	Upload, parse, chunk, embed, store
`POST /api/ask`	RAG question answering with sources
`POST /api/search`	Hybrid document search
`PUT /api/ai/settings/mode`	Switch between free trial and BYOK
`DELETE /api/admin/users/{id}`	Hard-delete a user and cascade Qdrant cleanup

Performance Engineering

A 600-page document produces roughly 2,300 chunks. Naively embedding one chunk per API call quickly runs into rate limits and latency. The system batches embedding calls, retries with backoff, and rate-limits registration to protect shared quota.



```java
 * a future upgrade, not needed right now.
 */
@Service
public class RateLimitService {

    @Value("${rate-limit.register.capacity}")
    private int capacity;

    @Value("${rate-limit.register.refill-minutes}")
    private int refillMinutes;

    // One bucket per IP address, kept in memory for the app's lifetime.
    private final Map<String, Bucket> buckets = new ConcurrentHashMap<>();

    /**
     * Returns true if the given key (e.g. an IP address) still
     * has tokens available, and consumes one token if so.
     * Returns false if the caller has exceeded their limit.
     */
    public boolean tryConsume(String key) {

        Bucket bucket = buckets.computeIfAbsent(key, k -> createNewBucket());

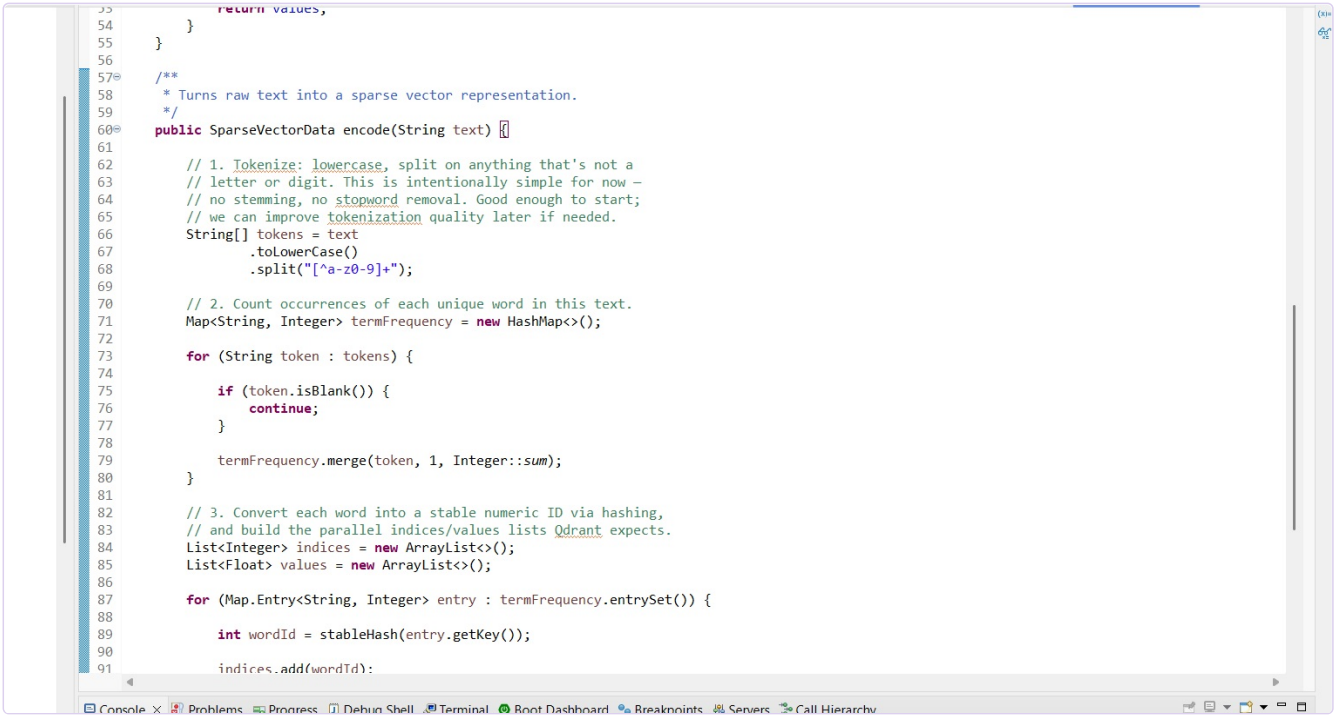
        return bucket.tryConsume(1);
    }

    private Bucket createNewBucket() {

        Bandwidth limit = Bandwidth.classic(
                capacity,
                Refill.greedy(capacity, Duration.ofMinutes(refillMinutes))
        );

        return Bucket.builder()
                .addLimit(limit)
                .build();
    }
}
```

RateLimitService.java — per-key token bucket, in-memory, configurable capacity and refill



```java
        }
    }

    /**
     * Turns raw text into a sparse vector representation.
     */
    public SparseVectorData encode(String text) {

        // 1. Tokenize: lowercase, split on anything that's not a
        // letter or digit. This is intentionally simple for now —
        // no stemming, no stopword removal. Good enough to start;
        // we can improve tokenization quality later if needed.
        String[] tokens = text
                .toLowerCase()
                .split("[^a-z0-9]+");

        // 2. Count occurrences of each unique word in this text.
        Map<String, Integer> termFrequency = new HashMap<>();

        for (String token : tokens) {

            if (token.isBlank()) {
                continue;
            }

            termFrequency.merge(token, 1, Integer::sum);
        }

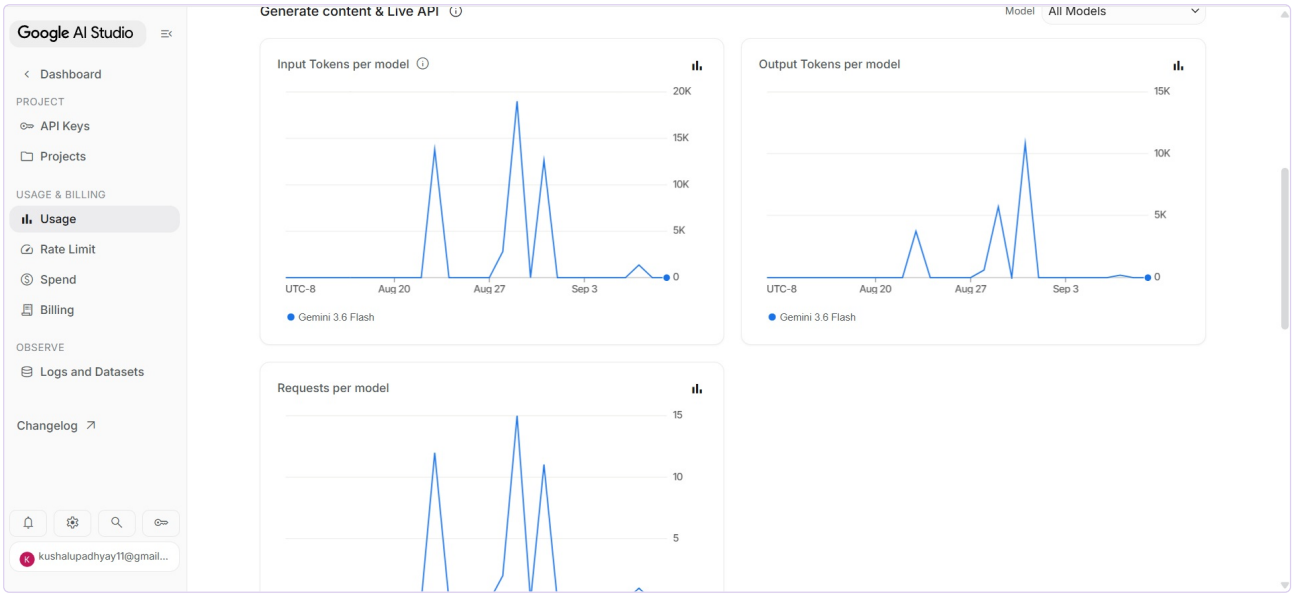
        // 3. Convert each word into a stable numeric ID via hashing,
        // and build the parallel indices/values lists Qdrant expects.
        List<Integer> indices = new ArrayList<>();
        List<Float> values = new ArrayList<>();

        for (Map.Entry<String, Integer> entry : termFrequency.entrySet()) {

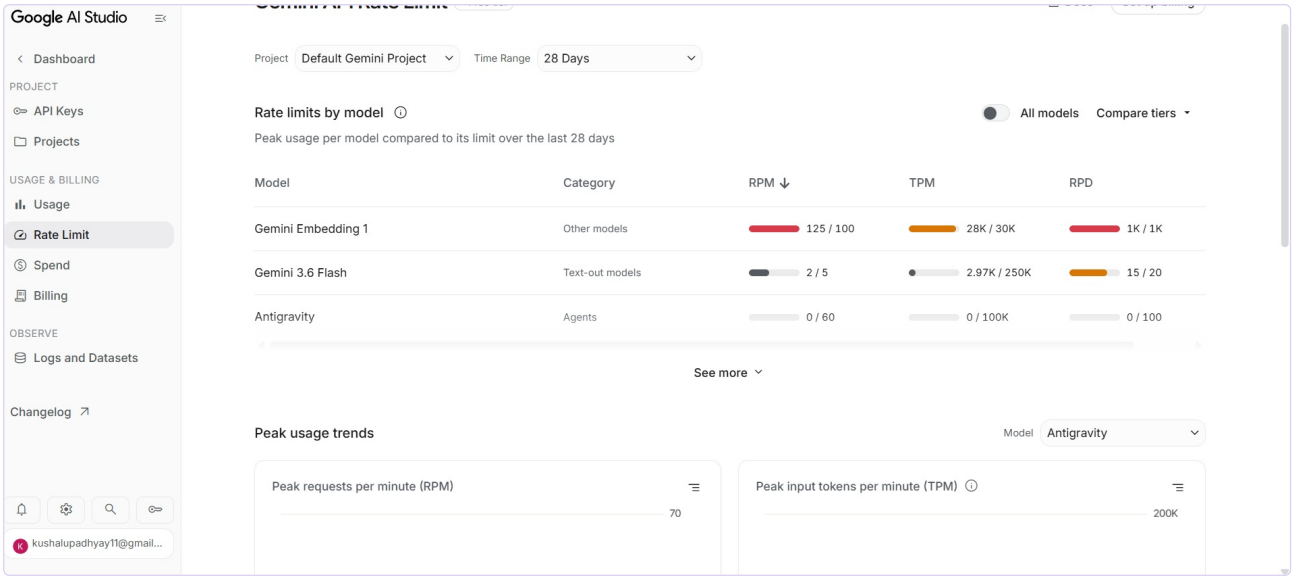
            int wordId = stableHash(entry.getKey());

            indices.add(wordId);
```

SparseVectorService.java — tokenize, count term frequency, hash into stable sparse indices



Google AI Studio — input/output token volume across development, by model



Google AI Studio — rate-limit headroom (RPM / TPM / RPD) per model

> Embedding batch size and concurrency are both externalised as configuration, so throughput can be tuned per-quota tier without a code change.

Engineering Decisions & Challenges

Why Qdrant instead of a plain vector index?

Native support for both dense and sparse vectors on the same point, with server-side payload filtering — needed for owner-scoped hybrid search without a second round trip.

Why PostgreSQL and Qdrant, not one database?

Relational data (users, ownership, settings) and high-dimensional vector data have different query patterns and different scaling needs; splitting them keeps each store doing what it's good at.

Why Reciprocal Rank Fusion?

RRF combines two rankings using only rank position, not raw scores from different scales — no manual tuning of a dense/sparse blend weight.

Challenge: Gemini embedding quota on large documents.

A 600-page document produced far more embedding requests than the free tier allowed. Fixed with sentence-aware chunking that grows for large documents, configurable batching, and retry-with-backoff.

Challenge: keeping one user's chunks out of another's search results.

Every chunk carries `ownerId` in its Qdrant payload, and every search query attaches a mandatory `ownerId` filter — enforced in the service layer, not left to the caller.

What I Learned

- Designing a hybrid retrieval system end to end — dense embeddings, sparse vectors, and rank fusion working together instead of in isolation.
- Multi-tenant data isolation that has to hold at the vector-database layer, not just the relational one.
- Spring Security with stateless JWT, including the tradeoffs of disabling CSRF for a token-based API.
- Encrypting and managing user-supplied secrets (AES-256-GCM) safely, including key rotation via enable/disable.
- Performance tuning under real API rate limits: batching, backoff, and adaptive chunking driven by production-shaped constraints, not just theory.

Future Improvements

> Not yet implemented — explicitly future work:

- Containerized / Dockerized deployment
- Cloud object storage for uploaded files (currently local disk)
- Horizontal scaling of the Spring Boot service
- Observability: metrics, tracing, structured logging pipeline
- Redis-backed distributed rate limiting (currently in-memory, single-instance)
- Additional LLM providers beyond Gemini

Summary

A full-stack, multi-tenant RAG platform combining document processing, hybrid vector retrieval, AI-generated answers grounded in cited sources, secure authentication, encrypted API-key management, and an administrative layer — built end to end in Java and React.

Java · Spring Boot · React · PostgreSQL · Qdrant · Gemini · JWT · Maven

github.com/kushal1-del · linkedin.com/in/kushalupadhyay108 · kushalupadhyay11@gmail.com · +91 85910 78291